

Lecture 7 Image Super Resolution

(Deep Learning Methods – Fully Supervised ideas)

Yuyao Zhang PhD

zhangyy8@shanghaitech.edu.cn

SIST Building-3 420

Image Super-Resolution

- Low resolution images can be modeled from high resolution images using the below formula, where $\mathbf{D}$ is the degradation function, $\mathbf{I}_y$ is the high resolution image, $\mathbf{I}_x$ is the low resolution image, and σ is the noise.

$$\mathbf{I}_x = \mathbf{D}(\mathbf{I}_y; \sigma)$$

- The degradation parameters $\mathbf{D}$ and σ are unknown; only the high resolution image and the corresponding low resolution image are provided. The task of the neural network is to find the inverse function of degradation using just the HR and LR image data.

Outline

- Interpolation methods
- Reconstruction based methods
- Deep learning based methods
 - Pre-Upsampling Super Resolution
 - Post-Upsampling Super Resolution
 - Residual Networks
 - Multi-Stage Residual Networks
 - Recursive Networks
 - Progressive Reconstruction Networks
 - Multi-Branch Networks
 - Attention-Based Networks
 - Generative Models

Presampling: SRCNN & VDSR

[1] Learning a Deep Convolutional Network for Image Super-Resolution (ECCV 2014)

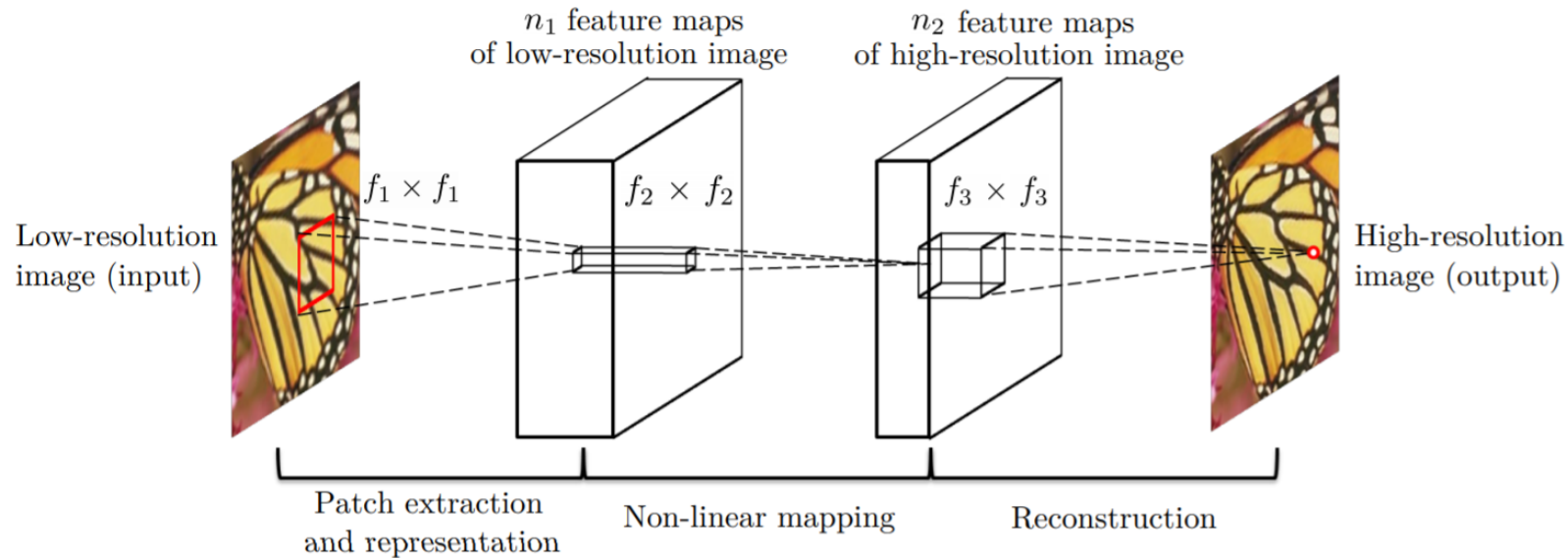
[2] Image Super-Resolution Using Deep Convolutional Networks (TPAMI 2015)



Pre-Upsampling Super Resolution

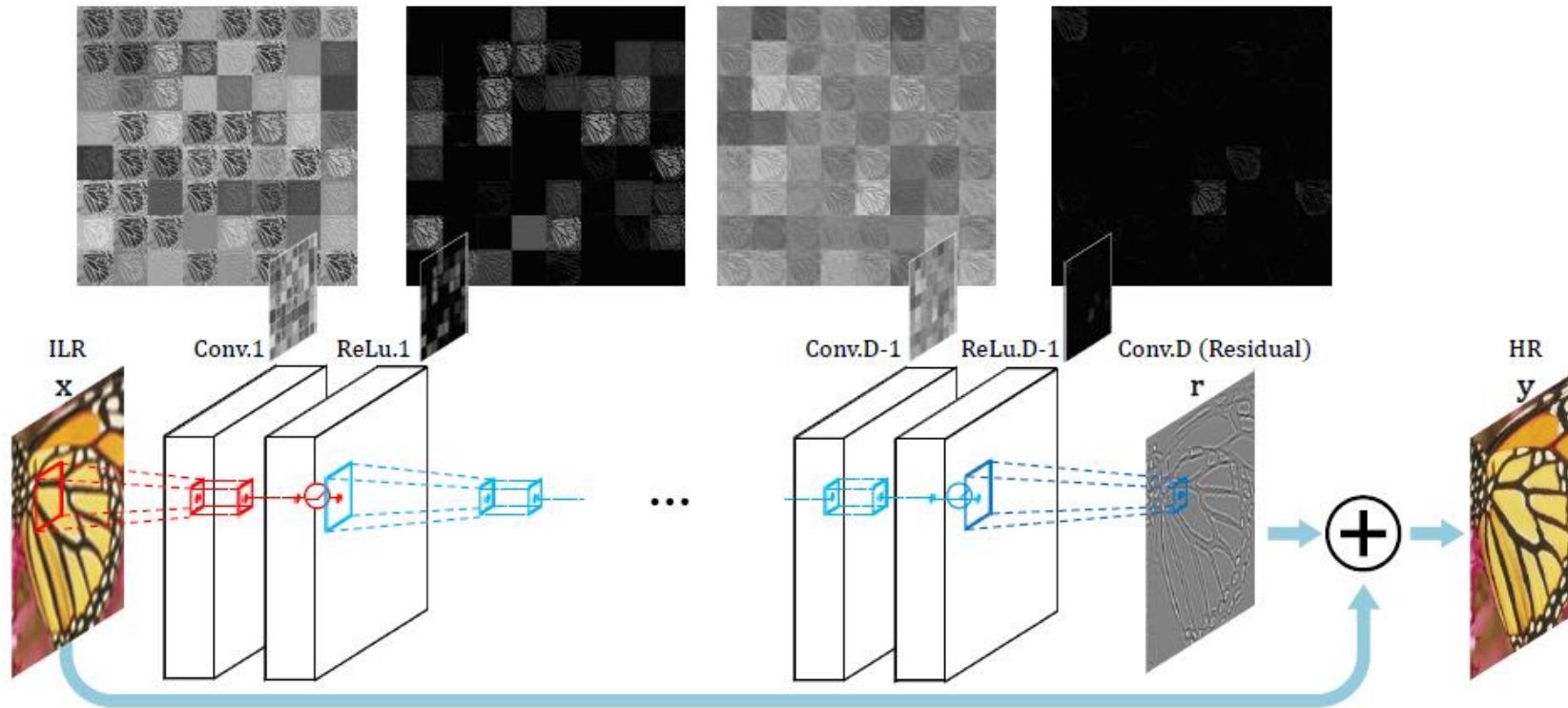
- The methods under this bracket use traditional techniques—like bicubic interpolation and deep learning—to refine an upsampled image.
- The most popular method, SRCNN, was also the first to use deep learning, and has achieved impressive results.

Image Super-Resolution Using Deep Convolutional Networks (SRCNN)



- SRCNN is a simple CNN architecture consisting of three layers:
 - Patch extraction
 - Non-linear mapping
 - Reconstruction

Very Deep Super Resolution (VDSR)



- Very Deep Super Resolution (VDSR) is an improvement on SRCNN

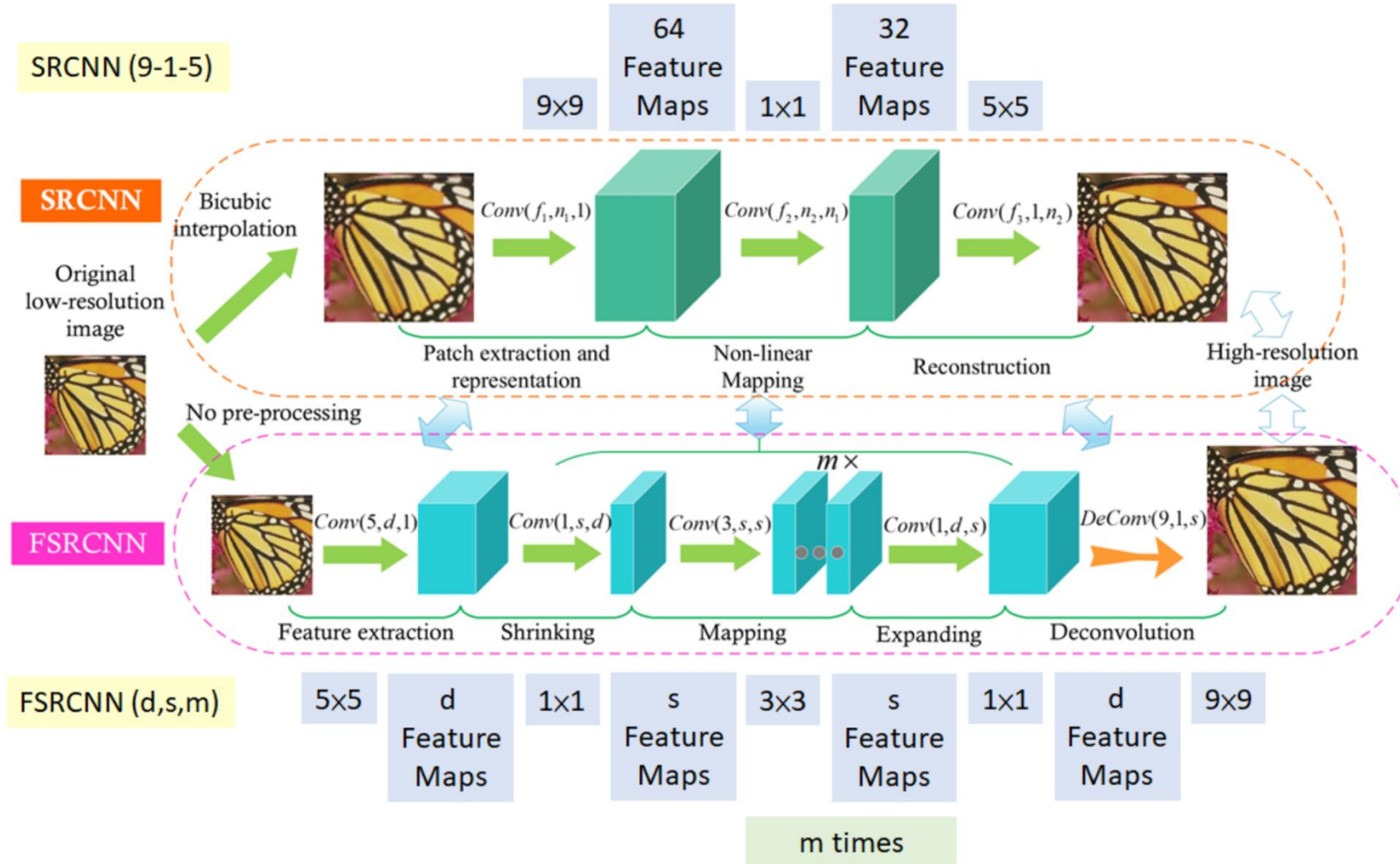
Post-Upsampling Super Resolution: FSRCNN & ESPCN

[1] Accelerating the Super-Resolution Convolutional Neural Networks (ECCV 2016)

[2] Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network (CVPR2016)



FSRCNN



FSRCNN

- The major changes between SRCNN and FSRCNN are:
 - There is no pre-processing or upsampling at the beginning. The feature extraction took place in the low resolution space.
 - A 1×1 convolution is used after the initial 5×5 convolution to reduce the number of channels, and hence lesser computation and memory, similar to how the Inception network is developed.
 - Multiple 3×3 convolutions are used, instead of having a big convolutional filter, similar to how the VGG network works by simplifying the architecture to reduce the number of parameters.
 - Upsampling is done by using a learned deconvolutional filter, thus improving the model.

Table 3. The results of PSNR (dB) and test time (sec) on three test datasets. All models are trained on the 91-image dataset.

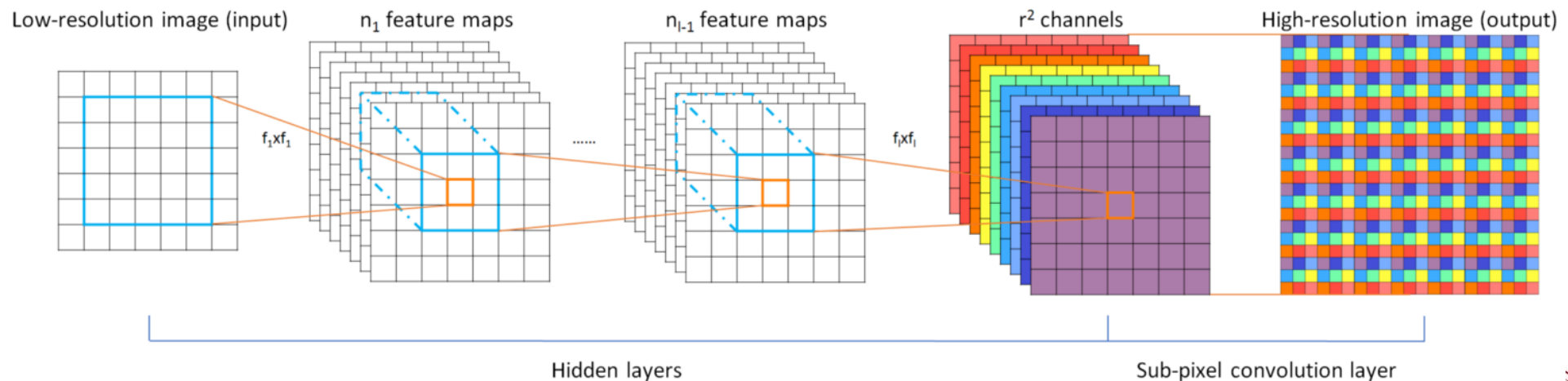
test dataset	upsampling factor	Bicubic		SRF [7]		SRCNN [1]		SRCNN-Ex [2]		SCN [8]		FSRCNN-s		FSRCNN	
		PSNR	Time	PSNR	Time	PSNR	Time	PSNR	Time	PSNR	Time	PSNR	Time	PSNR	Time
Set5	2	33.66	-	36.84	2.1	36.33	0.18	36.67	1.3	36.76	0.94	36.53	0.024	36.94	0.068
Set14	2	30.23	-	32.46	3.9	32.15	0.39	32.35	2.8	32.48	1.7	32.22	0.061	32.54	0.16
BSD200	2	29.70	-	31.57	3.1	31.34	0.23	31.53	1.7	31.63	1.1	31.44	0.033	31.73	0.098
Set5	3	30.39	-	32.73	1.7	32.45	0.18	32.83	1.3	33.04	1.8	32.55	0.010	33.06	0.027
Set14	3	27.54	-	29.21	2.5	29.01	0.39	29.26	2.8	29.37	3.6	29.08	0.023	29.37	0.061
BSD200	3	27.26	-	28.40	2.0	28.27	0.23	28.47	1.7	28.54	2.4	28.32	0.013	28.55	0.035
Set5	4	28.42	-	30.35	1.5	30.15	0.18	30.45	1.3	30.82	1.2	30.04	0.0052	30.55	0.015
Set14	4	26.00	-	27.41	2.1	27.21	0.39	27.44	2.8	27.62	2.3	27.12	0.0099	27.50	0.029
BSD200	4	25.97	-	26.85	1.7	26.72	0.23	26.88	1.7	27.02	1.4	26.73	0.0072	26.92	0.019

ESPCN

- ESPCN introduces the concept of sub-pixel convolution to replace the deconvolutional layer for upsampling. This solves two problems associated with it:
 - Deconvolution happens in the high resolution space, and thus is more computationally expensive.
 - It resolves the checkerboard issue in deconvolution, which occurs due to the overlap operation of convolution (shown below).

ESPCN

- Sub-pixel convolution works by converting depth to space, as seen in the figure below. Pixels from multiple channels in a low resolution image are rearranged to a single channel in a high resolution image. To give an example, an input image of size $5 \times 5 \times 4$ can rearrange the pixels in the final four channels to a single channel, resulting in a 10×10 HR image.



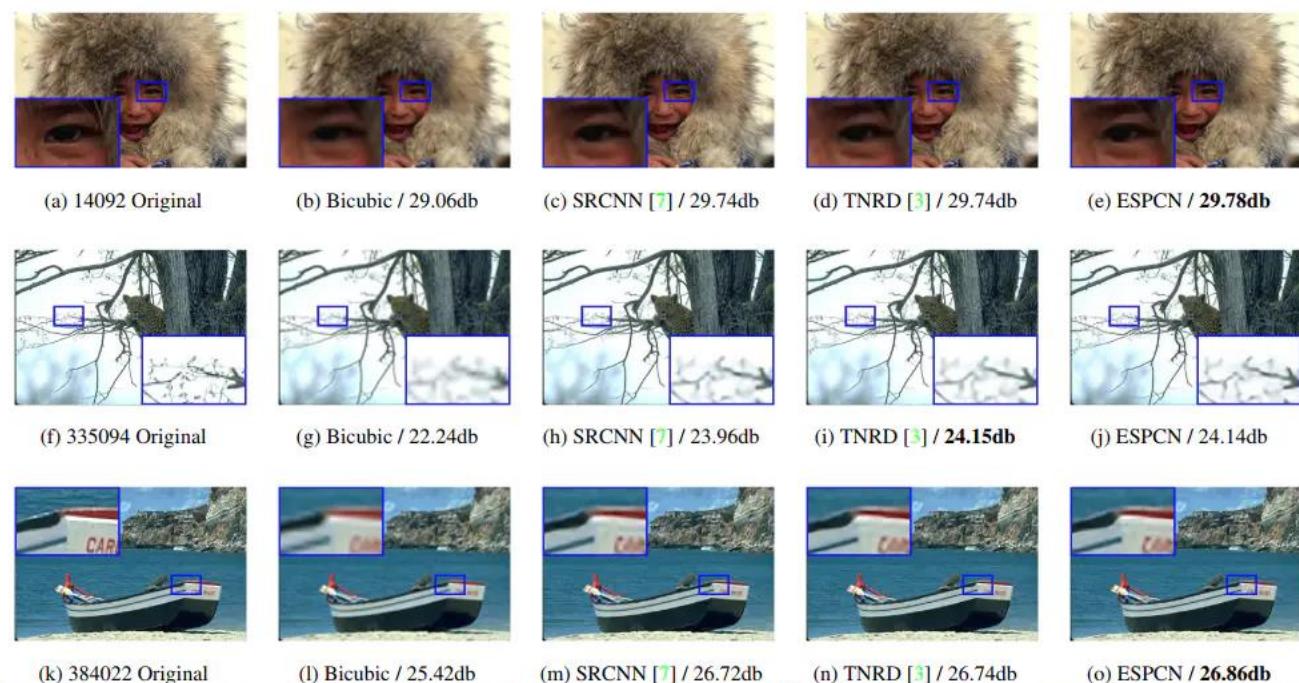


Figure 6. Super-resolution examples for "14092", "335094" and "384022" from **BSD500** with an upscaling factor of 3. PSNR values are shown under each sub-figure.

https://blog.csdn.net/xu_fu_yong

Dataset	Scale	SRCNN (91)	ESPCN (91 <i>relu</i>)	ESPCN (91)	SRCNN (ImageNet)	ESPCN (ImageNet <i>relu</i>)
Set5	3	32.39	32.39	32.55	32.52	33.00
Set14	3	29.00	28.97	29.08	29.14	29.42
BSD300	3	28.21	28.20	28.26	28.29	28.52
BSD500	3	28.28	28.27	28.34	28.37	28.62
SuperTexture	3	26.37	26.38	26.42	26.41	26.69
Average	3	27.76	27.76	27.82	27.83	28.09

Table 1. The mean PSNR (dB) for different models. Best results for each category are shown in bold. There is significant difference between the PSNRs of the proposed method and other methods (p -value < 0.001 with paired t-test).

https://blog.csdn.net/xu_fu_yong

Post-Upsampling Super-Resolution

- Since the feature extraction process in pre-upsampling SR occurs in the high resolution space, the computational power required is also on the higher end. Post-upsampling SR tries to solve this by doing feature extraction in the lower resolution space, then doing upsampling only at the end, therefore significantly reducing computation.
- Also, instead of using simple bicubic interpolation for upsampling, a learned upsampling in the form of deconvolution/sub-pixel convolution is used, thus making the network trainable end-to-end.

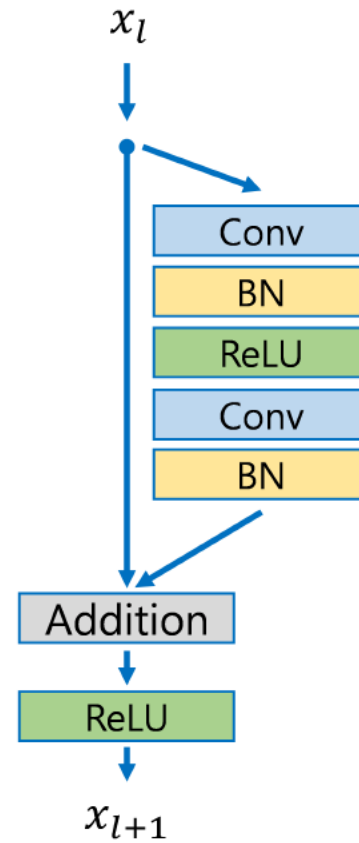
Residual Networks: EDSR & MDSR & CARN

[1] EDSR & MDSR Enhanced Deep Residual Networks for Single Image Super-Resolution (CVPR 2017)

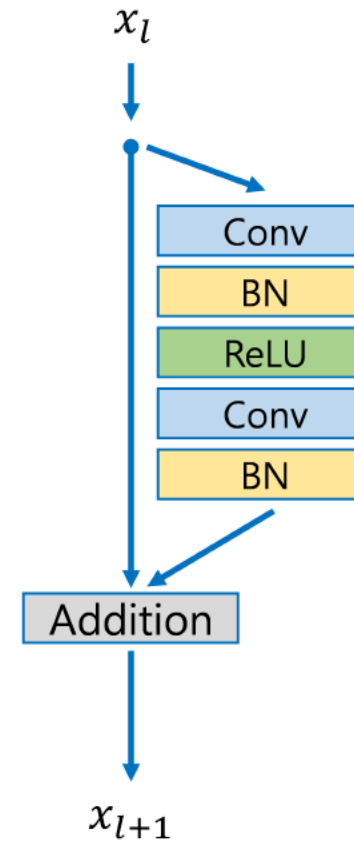
[2] CARN: Fast, Accurate, and Lightweight Super-Resolution with Cascading Residual Network (ECCV 2018)

EDSR

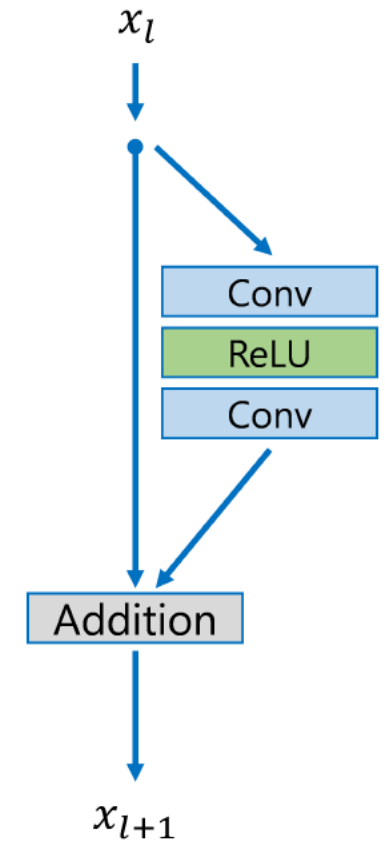
- The EDSR architecture is based on the SRResNet architecture, consisting of multiple residual blocks.
- The major difference from SRResNet is that the Batch Normalization layers are removed. The author states that BN normalizes the input, thus limiting the range of the network; removal of BN results in an improvement in accuracy.



(a) Original



(b) SRResNet



(c) Proposed

EDSR

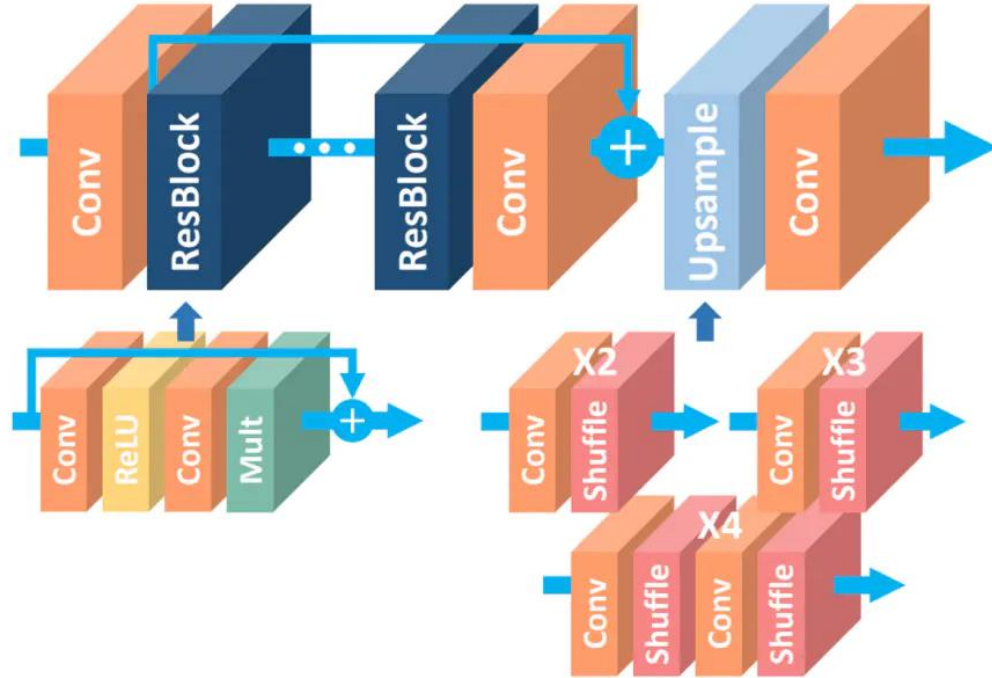


Figure 3: The architecture of the proposed single-scale SR network (EDSR).

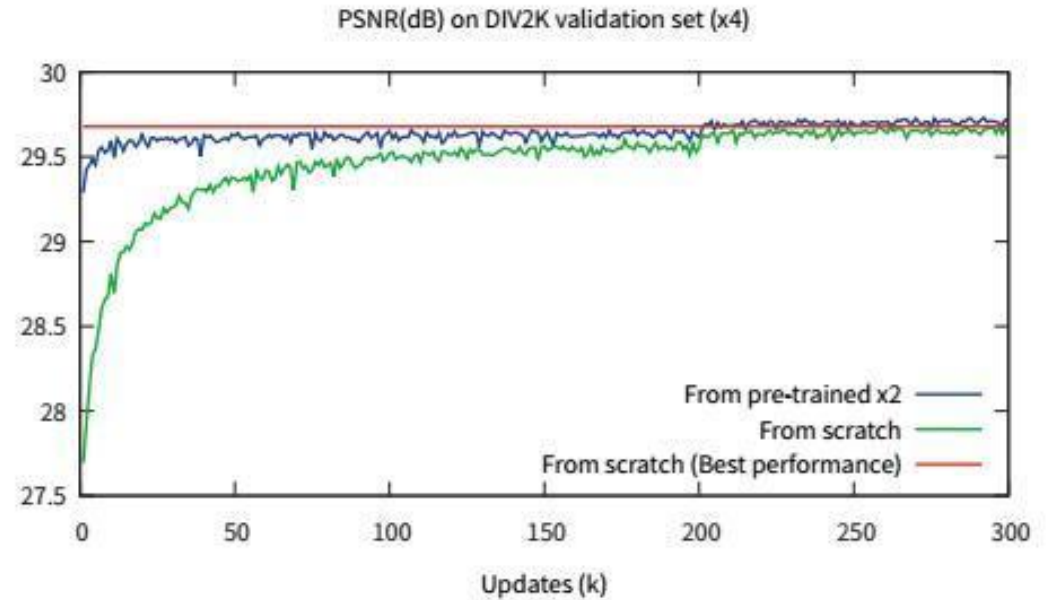


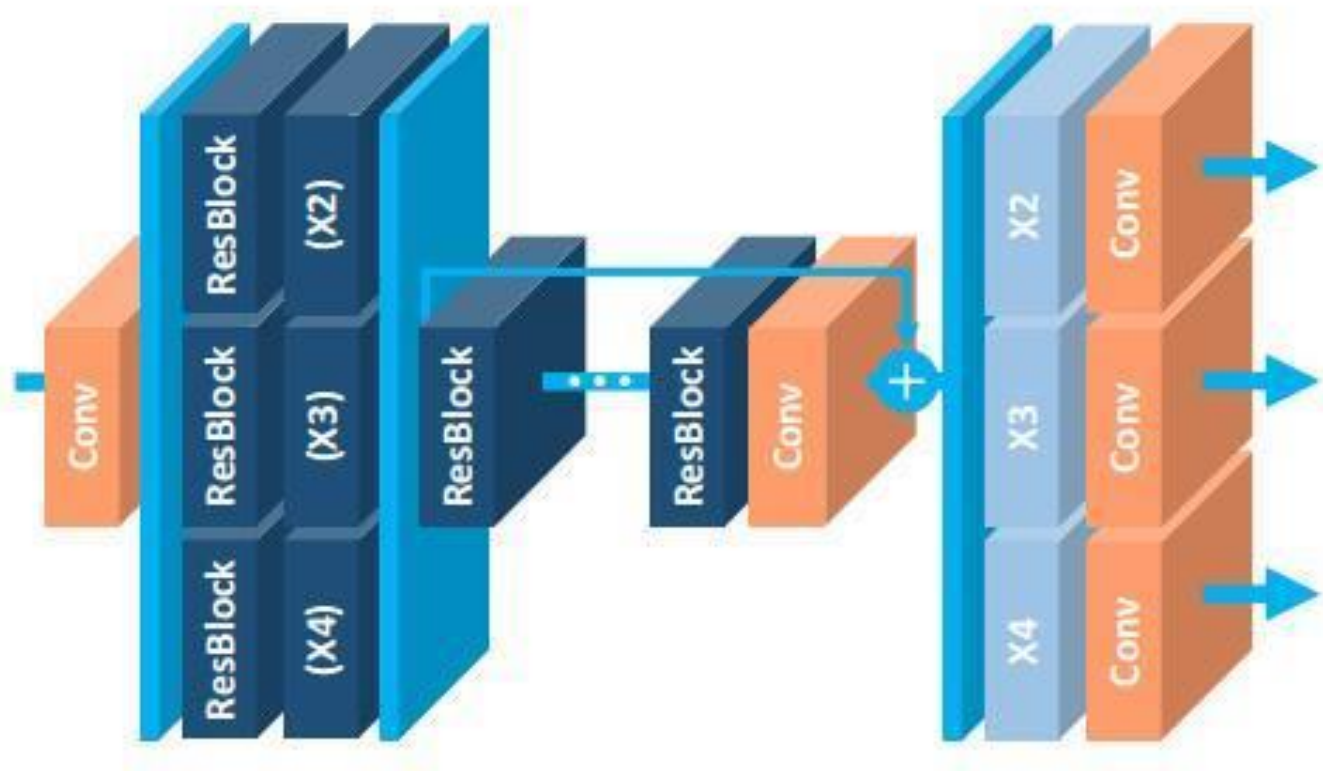
Figure 4: Effect of using pre-trained $\times 2$ network for $\times 4$ model (EDSR). The red line indicates the best performance of green line. 10 images are used for validation during training.

http://blog.csdn.net/shwan_ma

MDSR

- MDSR is an extension of EDSR, with multiple input and output modules that give corresponding resolution outputs at 2x, 3x, and 4x.
- At the beginning, the pre-processing modules for scale-specific input are present consisting of two residual blocks with 5×5 kernels. A large kernel is used for the pre-processing layers to keep the network shallow, while still achieving a high receptive field.
- At the end of the scale-specific pre-processing modules are the shared residual blocks, which is a common block for data of all resolutions. Finally, after the shared residual blocks are the scale-specific upsampling modules.

MDSR

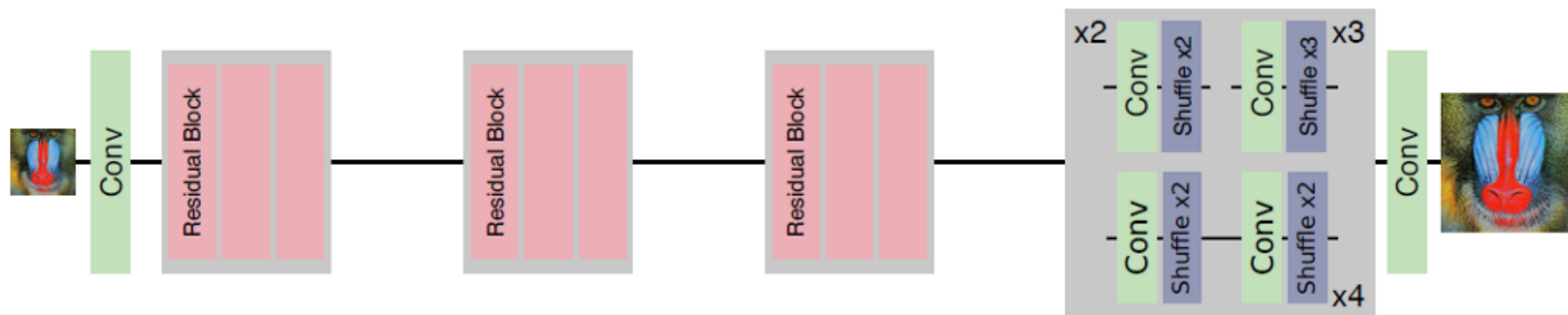


Scale	SRResNet (L2 loss)	SRResNet (L1 loss)	Our baseline (Single-scale)	Our baseline (Multi-scale)	EDSR (Ours)	MDSR (Ours)	EDSR+ (Ours)	MDSR+ (Ours)
×2	34.40 / 0.9662	34.44 / 0.9665	34.55 / 0.9671	34.60 / 0.9673	35.03 / 0.9695	34.96 / 0.9692	35.12 / 0.9699	35.05 / 0.9696
×3	30.82 / 0.9288	30.85 / 0.9292	30.90 / 0.9298	30.91 / 0.9298	31.26 / 0.9340	31.25 / 0.9338	31.39 / 0.9351	31.36 / 0.9346
×4	28.92 / 0.8960	28.92 / 0.8961	28.94 / 0.8963	28.95 / 0.8962	29.25 / 0.9017	29.26 / 0.9016	29.38 / 0.9032	29.36 / 0.9029

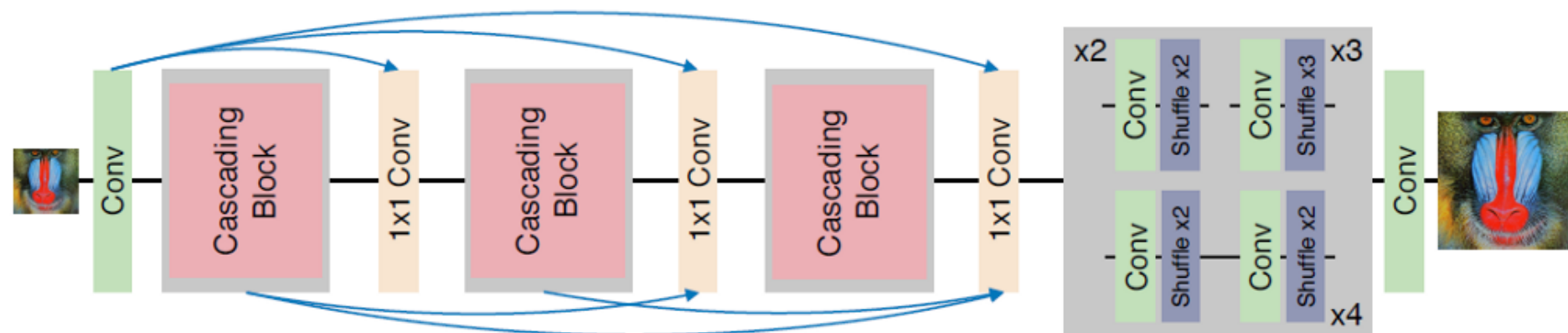
CARN

- In the paper, the authors have proposed the following advancements on top of a traditional residual network:
- A cascading mechanism at both the local and global level, to incorporate features from multiple layers and give the network the ability to receive more information.
- In addition to CARN, a smaller CARN-M is proposed to have a lighter architecture, without much deterioration in results, with the help of recursive network architecture.

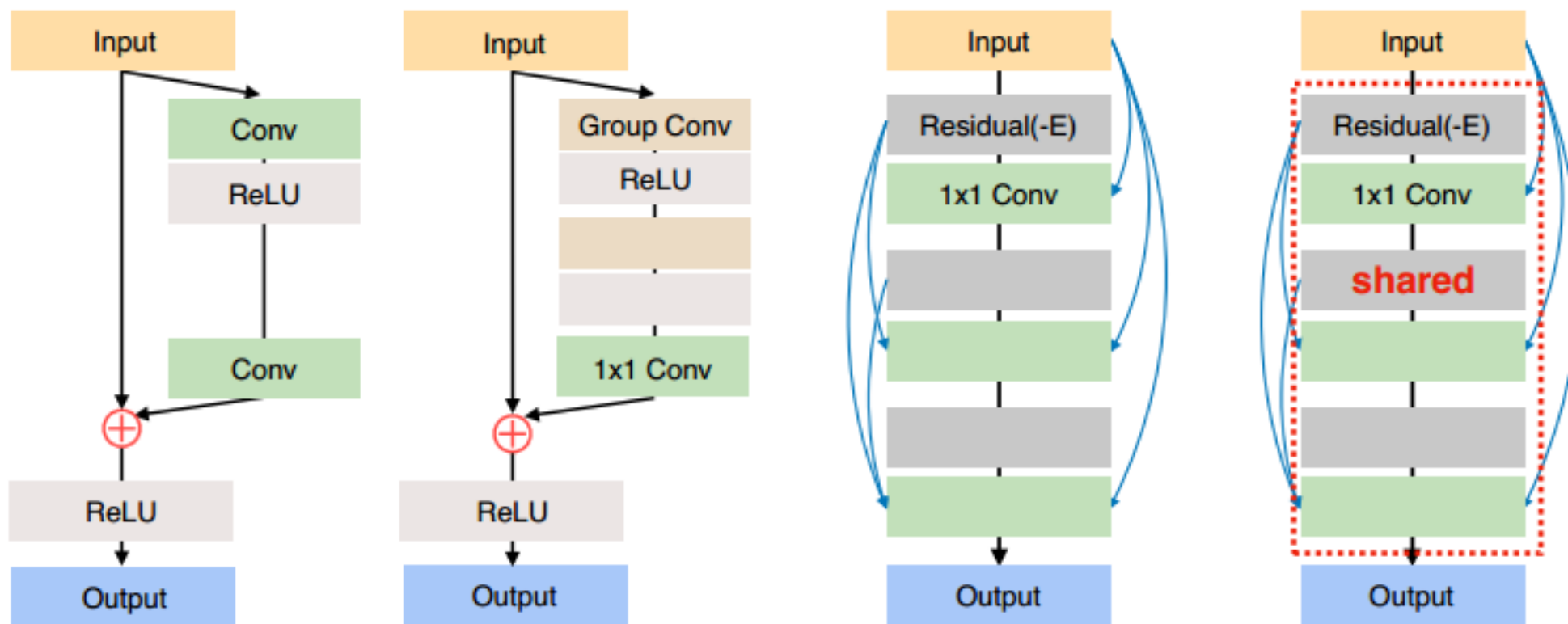
CARN



(a) Plain ResNet



(b) Cascading Residual Network (CARN)



(a) Residual Block (b) Residual-E Block (c) Cascading Block (d) Recursive Block

Fig. 3: Simplified structures of (a) residual block (b) efficient residual block (residual-E), (c) cascading block and (d) recursive cascading block. The $\oplus$ operations in (a) and (b) are element-wise addition for residual learning.

Multi-Stage Residual Networks: BTSRN

[1] EDSR & MDSR Balance two-stage Residual Networks for Image Super-Resolution (CVPR-W 2017)

Multi-Stage Residual Networks

- To deal with the task of feature extraction separately in the low-resolution space and high-resolution space, a multi-stage design is considered in a few architectures to improve their performance. The first stage predicts the coarse features, while the later stage improves on it. Let's discuss an architecture involving one of these multi-stage networks.

BTSRN

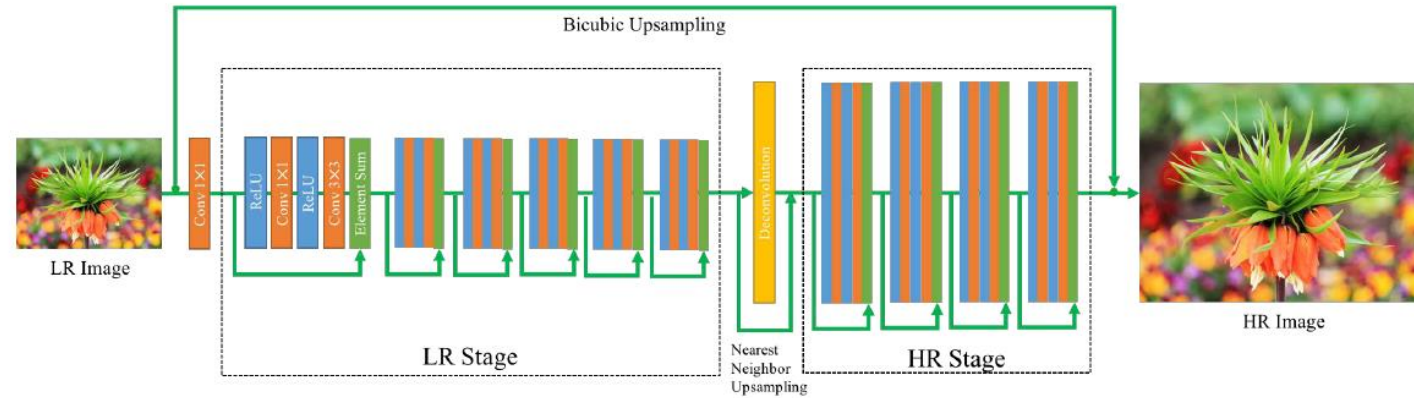
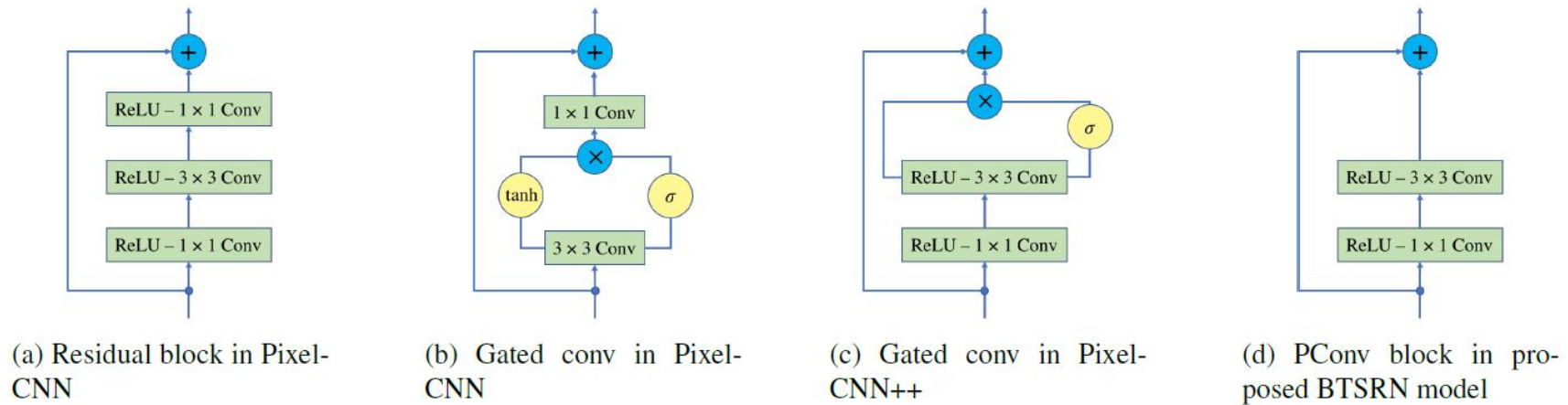


Figure 1: Architecture of the proposed network



BTSRN

- BTSRN consists of two stages: a low resolution (LR) stage and a high resolution (HR) stage.
- The LR stage consists of 6 residual blocks, whereas the HR stage contains 4 residual blocks. Convolution in the HR stage requires more computation than in the LR stage, as the input size is higher.
- The number of blocks in both the stages are determined in such a way as to achieve a trade-off between accuracy and performance.

BTSRN

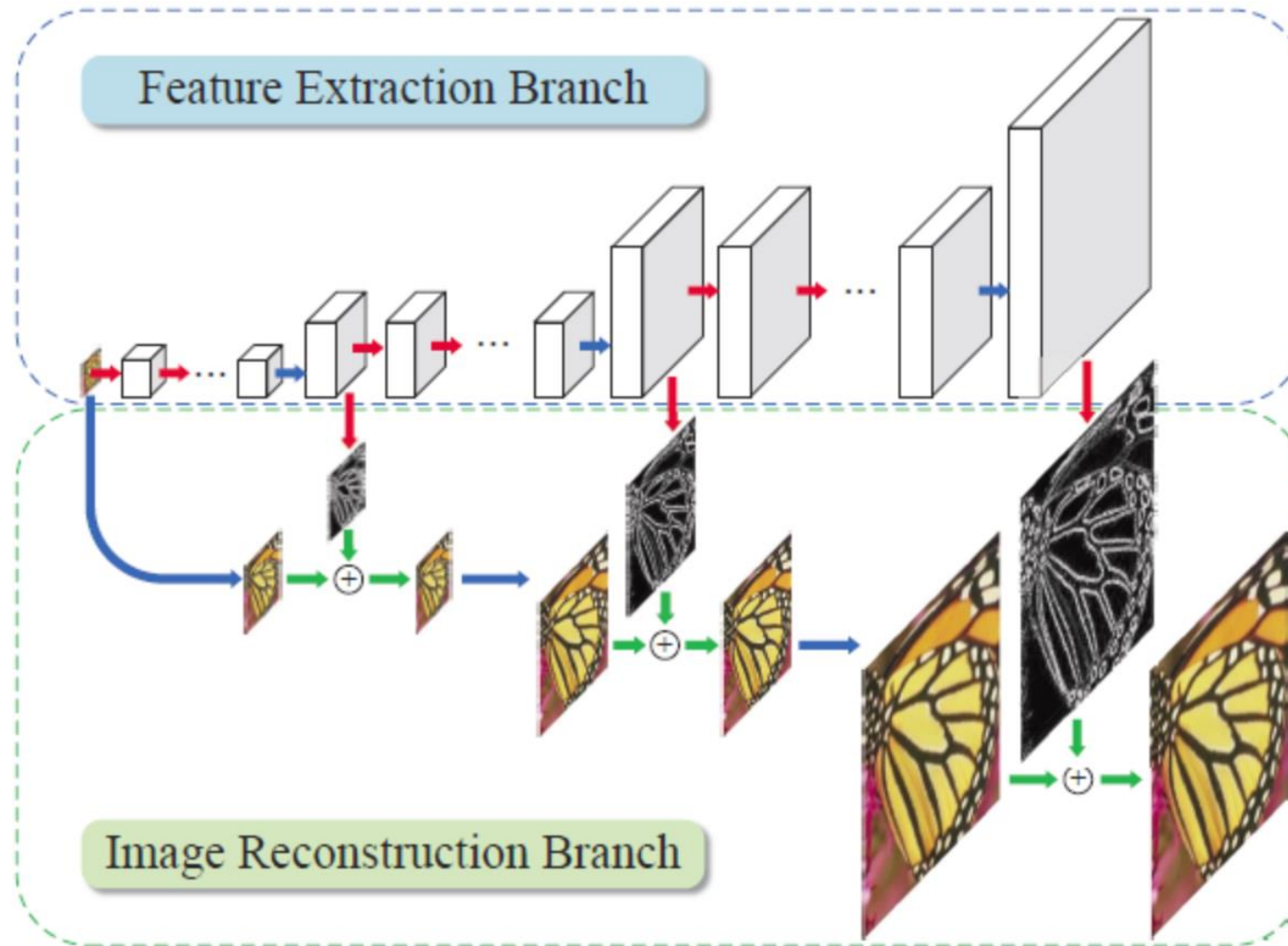
- BTSRN consists of two stages: a low resolution (LR) stage and a high resolution (HR) stage.
- The LR stage consists of 6 residual blocks, whereas the HR stage contains 4 residual blocks. Convolution in the HR stage requires more computation than in the LR stage, as the input size is higher.
- The number of blocks in both the stages are determined in such a way as to achieve a trade-off between accuracy and performance.

Progressive Reconstruction Networks: LAPSRN

[1] EDSR & MDSR Balance two-stage Residual Networks for Image Super-Resolution (CVPR-W 2017)

[2] CARN: Fast, Accurate, and Lightweight Super-Resolution with Cascading Residual Network (ECCV 2018)

LAPSRN

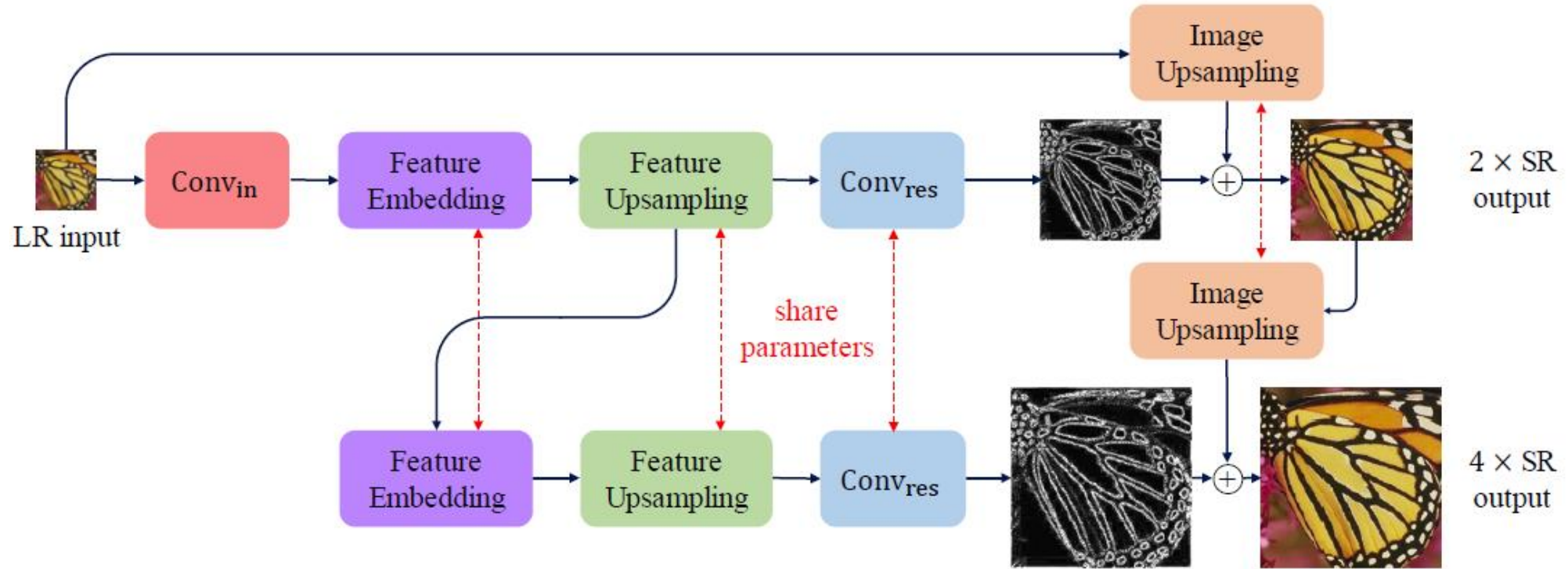


LAPSRN

- LAPSRN, or MS-LAPSRN, consists of a Laplacian pyramid structure which can upscale images to 2x, 4x, and 8x using a step-by-step approach.
- LAPSRN consists of multiple stages. The network consists of two branches: the Feature Extraction Branch and the Image Reconstruction Branch.

LAPSRN

- Recursive block



Attention-Based Networks: SelNet & RCAN

[1] RCAN Image Super-Resolution Using Very Deep Residual Channel Attention Networks (ECCV 2018)

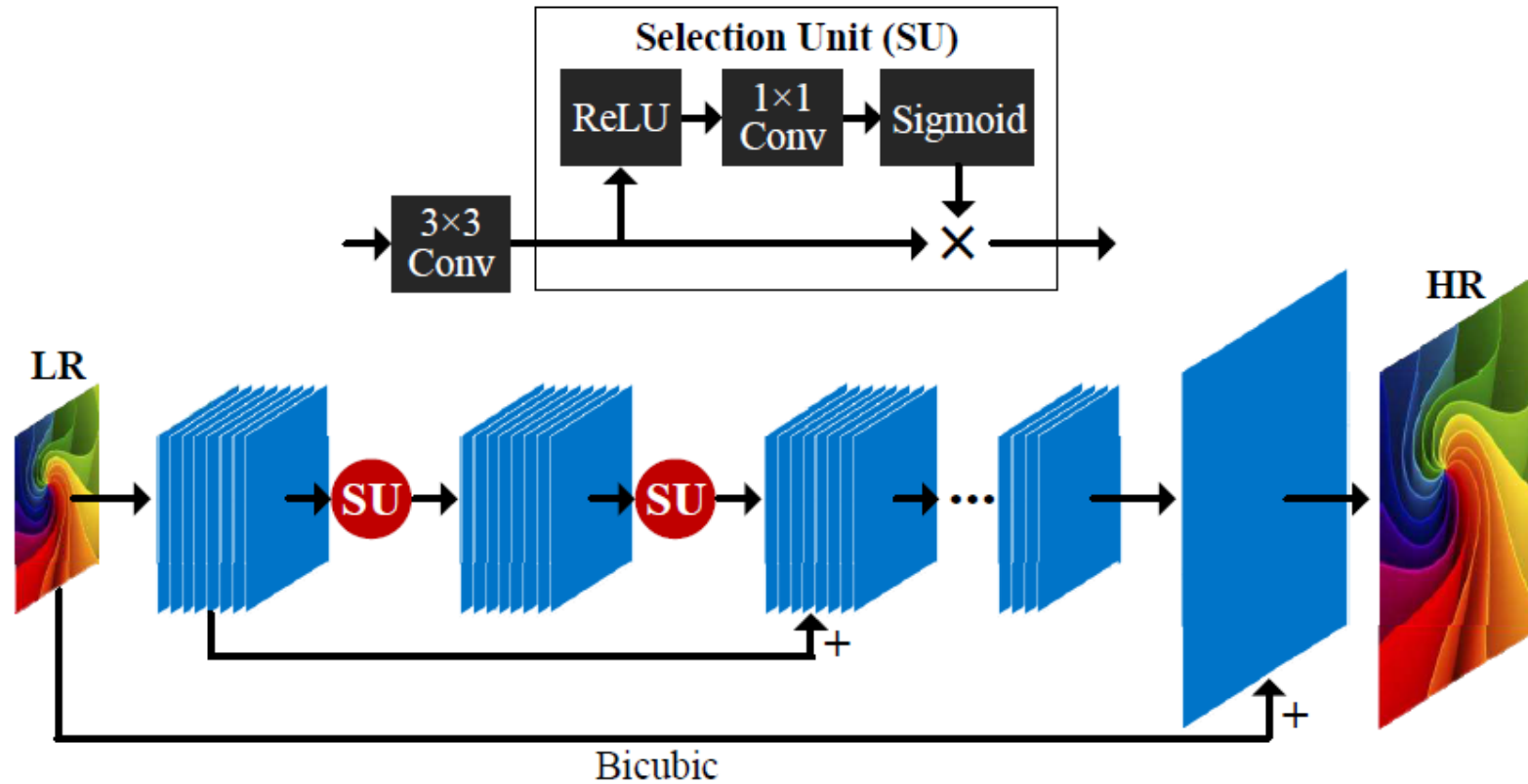
[2] SelNet: Fast, Accurate, and Lightweight Super-Resolution with Cascading Residual Network (ECCV 2018)

Attention-Based Networks

- The networks discussed so far give equal importance to all spatial locations and channels. In general, giving selective attention to different regions in an image can give much better results. We shall now discuss few architectures which help in achieving this.

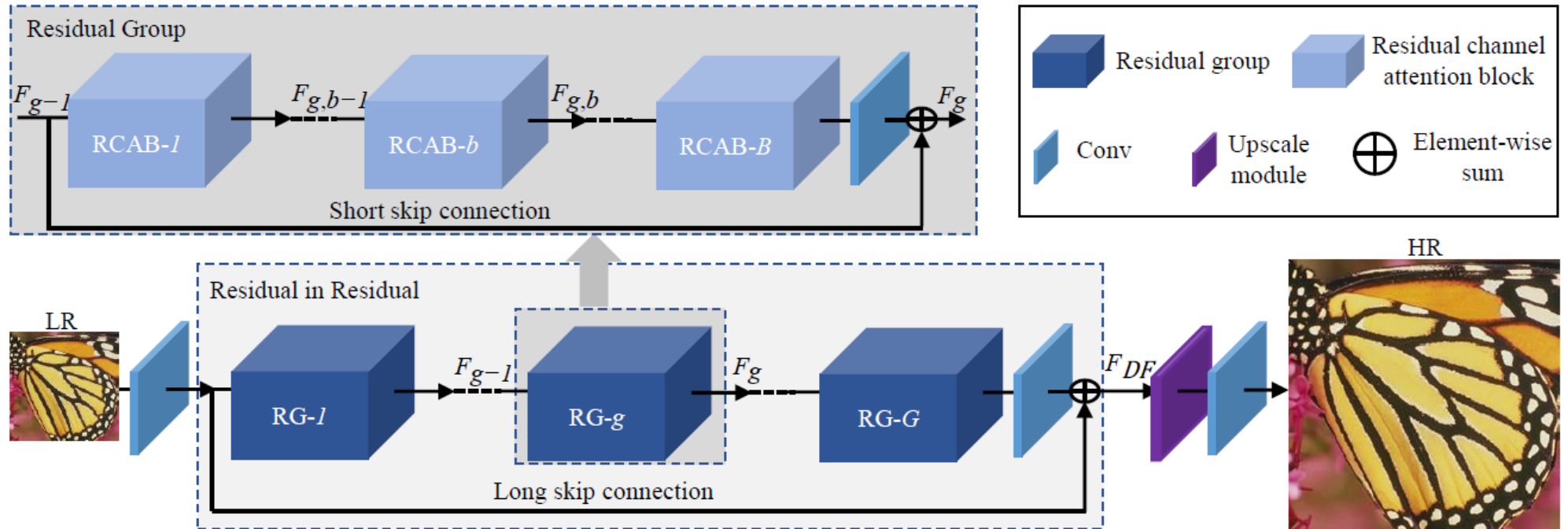
SelNet

- Residual in Residual (RIR)



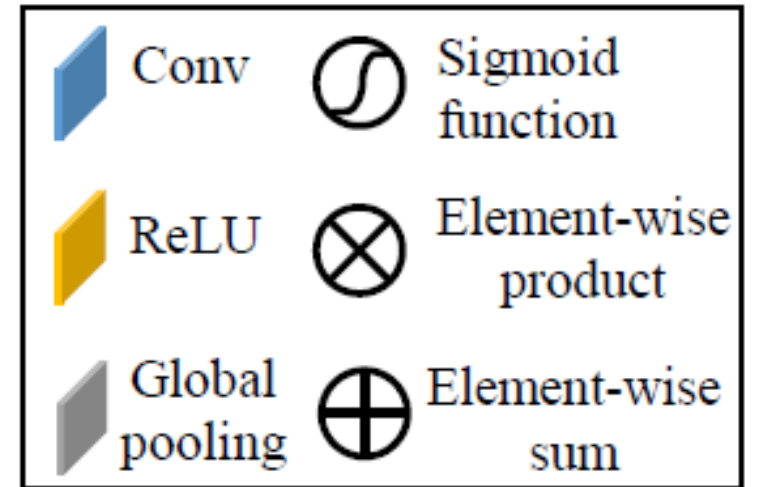
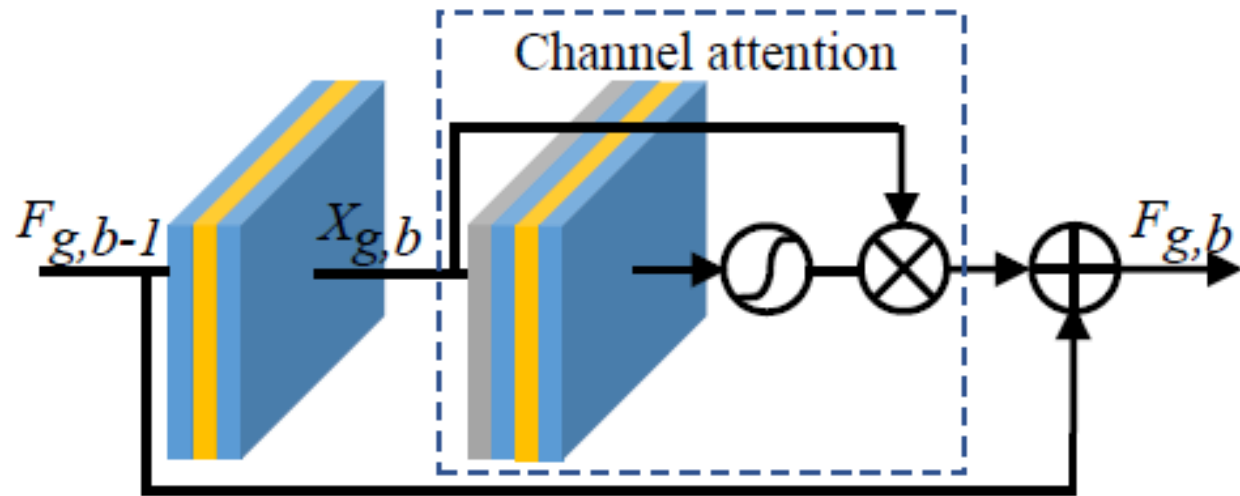
RCAN

- Residual in Residual (RIR)



RCAN

- Channel Attention (CA)



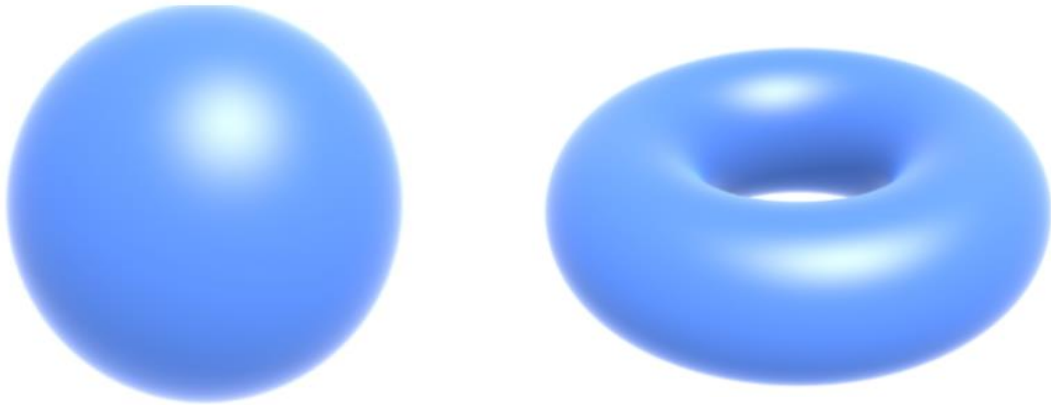
Implicit Representation SR: IREM & LIIF

- [1] IREM: High-Resolution Magnetic Resonance Image Reconstruction via Implicit Neural Representation (MICCAI 2021)
- [2] LIIF : Learning Continuous Image Representation with Local Implicit Image Function (CVPR 2021)

Latest trend: Implicit vs Explicit

Q1: $f(x, y, z) = x^2 + y^2 + z^2 - 1$

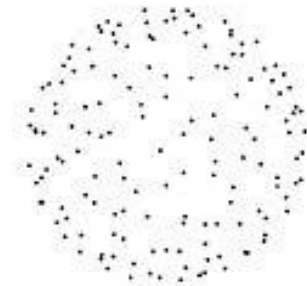
Q2: $f(x, y, z) = (2 - \sqrt{x^2 + y^2})^2 + z^2 - 1$



Implicit representation of a surface: Continuous

All points are **given directly** or **via parameter mapping**

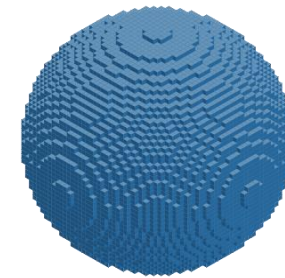
Generally: $f : \mathbb{R}^2 \rightarrow \mathbb{R}^3; (u, v) \mapsto (x, y, z)$



Point cloud



Mesh

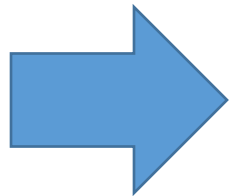


Voxel

Explicit representation of a surface: Discrete



How to present this using implicit surface?

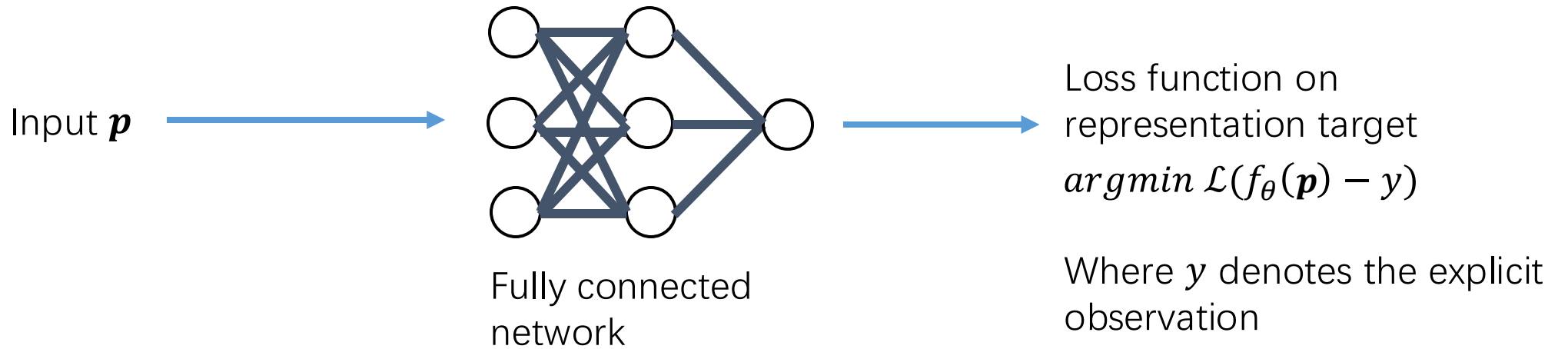


$$f(x, y, z) = ?$$

How about using DNN?



Structure of implicit neural representation



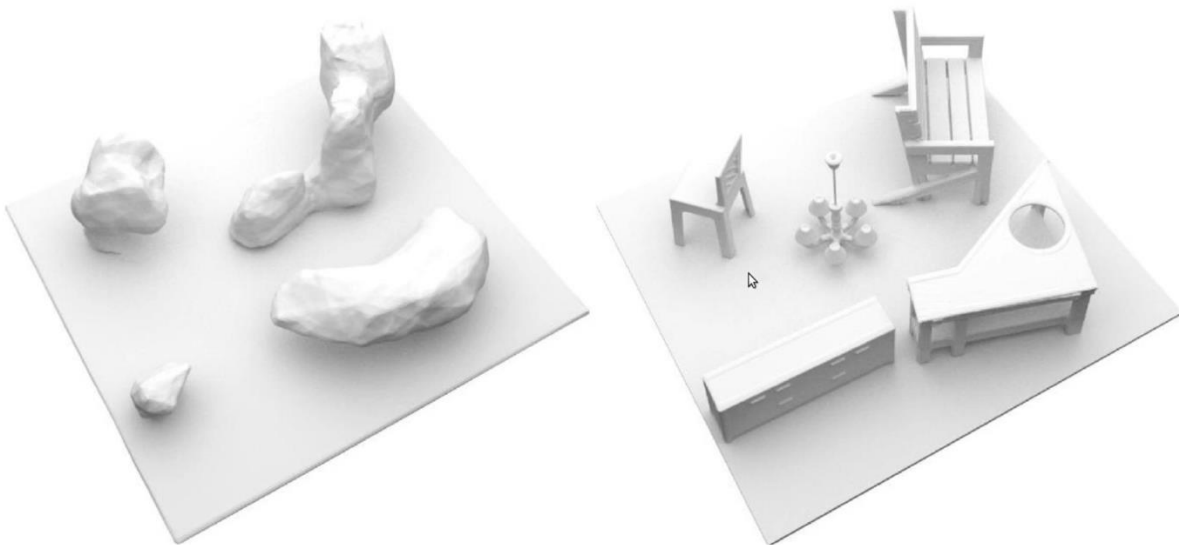
- Images: $f: \mathbb{R}^3 \rightarrow \mathbb{R}^3, f(x, y, z) = (R, G, B)$.
- Videos: $f: \mathbb{R}^3 \rightarrow \mathbb{R}^3, f(x, y, t) = (R, G, B)$.
- Shapes: $f: \mathbb{R}^3 \rightarrow \mathbb{R}^1, f(x, y, z) = 0 \text{ or } 1 \text{ as occupancy}$.
- Limitations:
 - Slow for training, need very deep structure.
 - Only present low-frequency information and small scenes.

Limitations

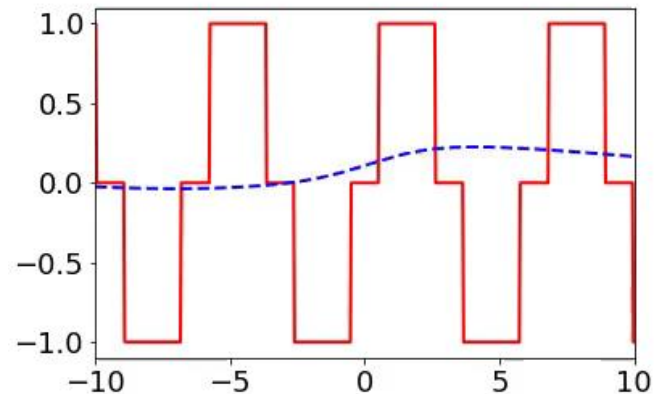
- OK for simple and small scene representation.



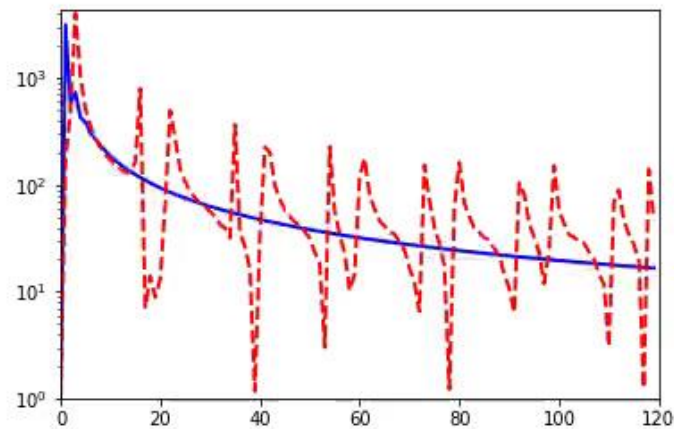
- Limited performance on large scene representation.



- Frequency Principle:



Red:
target function
Blue:
DNN fitting



Implicit Neural Representations with Periodic Activation Functions

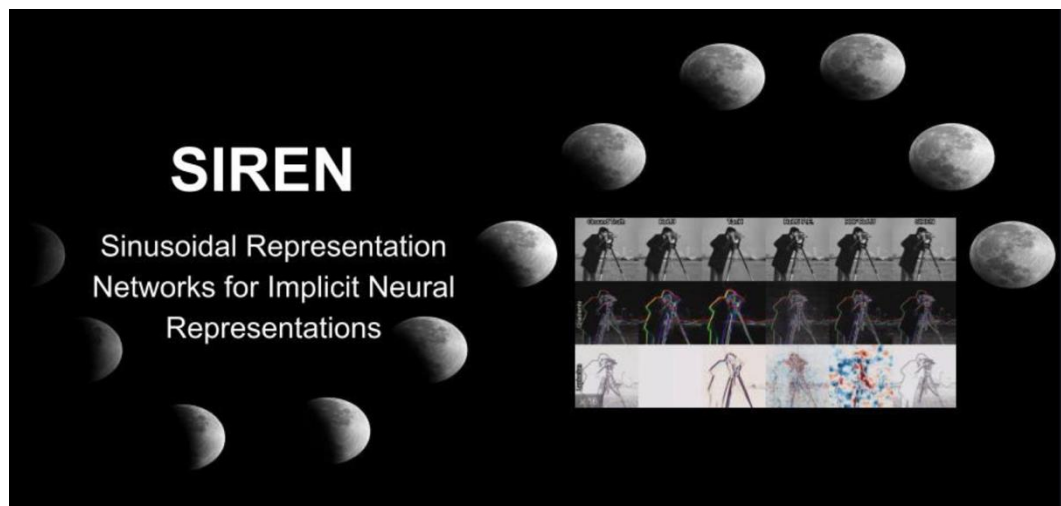
Vincent Sitzmann*
sitzmann@cs.stanford.edu

Julien N. P. Martel*
jnmartel@stanford.edu

Alexander W. Bergman
awb@stanford.edu

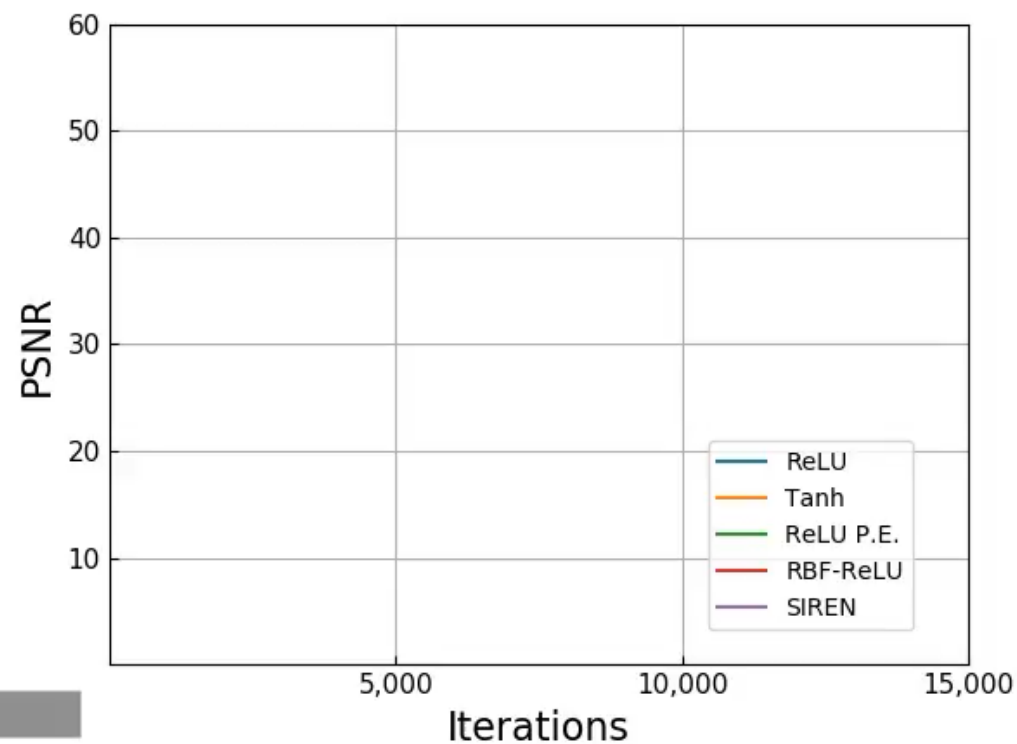
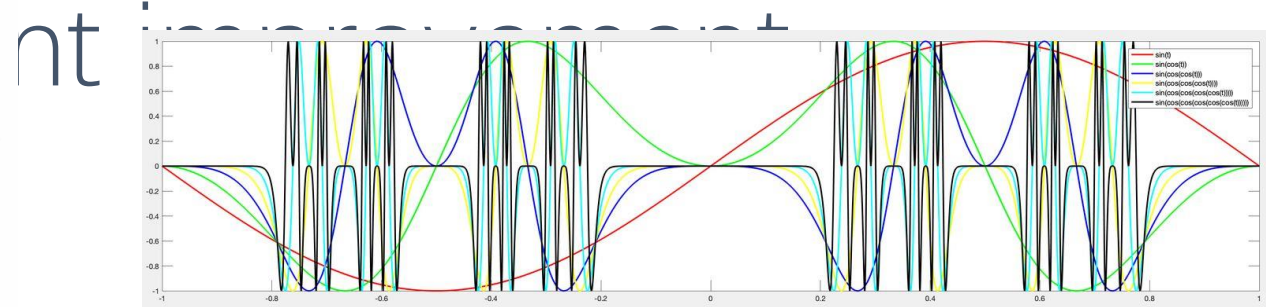
David B. Lindell
lindell@stanford.edu

Gordon Wetzstein
gordon.wetzstein@stanford.edu



$$\Phi(\mathbf{x}) = \mathbf{W}_n (\phi_{n-1} \circ \phi_{n-2} \circ \dots \circ \phi_0)(\mathbf{x}) + \mathbf{b}_n, \quad \mathbf{x}_i \mapsto \phi_i(\mathbf{x}_i) = \sin(\mathbf{W}_i \mathbf{x}_i + \mathbf{b}_i). \quad (4)$$

Here, $\phi_i : \mathbb{R}^{M_i} \mapsto \mathbb{R}^{N_i}$ is the i^{th} layer of the network. It consists of the affine transform defined by the weight matrix $\mathbf{W}_i \in \mathbb{R}^{N_i \times M_i}$ and the biases $\mathbf{b}_i \in \mathbb{R}^{N_i}$ applied on the input $\mathbf{x}_i \in \mathbb{R}^{M_i}$, followed by the sine nonlinearity applied to each component of the resulting vector.



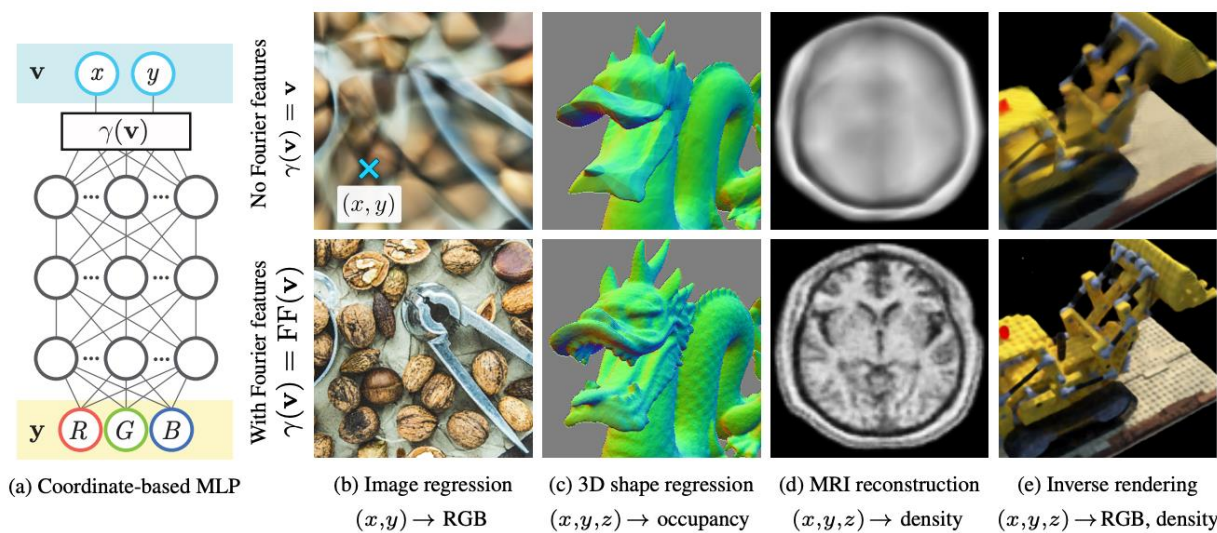
Several recent efficient ir

Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains

Matthew Tancik^{1*} Pratul P. Srinivasan^{1,2*} Ben Mildenhall^{1*} Sara Fridovich-Keil¹

Nithin Raghavan¹ Utkarsh Singhal¹ Ravi Ramamoorthi³ Jonathan T. Barron² Ren Ng¹

¹University of California, Berkeley ²Google Research ³University of California, San Diego

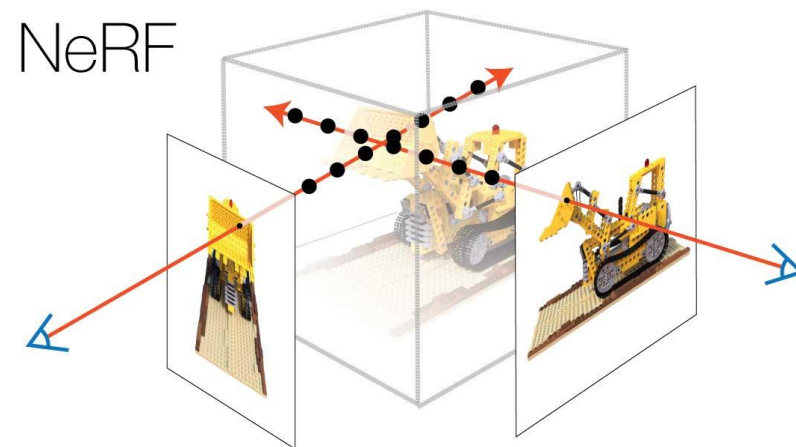


$$\gamma(\mathbf{v}) = [a_1 \cos(2\pi \mathbf{b}_1^T \mathbf{v}), a_1 \sin(2\pi \mathbf{b}_1^T \mathbf{v}), \dots, a_m \cos(2\pi \mathbf{b}_m^T \mathbf{v}), a_m \sin(2\pi \mathbf{b}_m^T \mathbf{v})]^T$$

NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis

Ben Mildenhall^{1*} Pratul P. Srinivasan^{1*} Matthew Tancik^{1*}
Jonathan T. Barron² Ravi Ramamoorthi³ Ren Ng¹

¹UC Berkeley ²Google Research ³UC San Diego

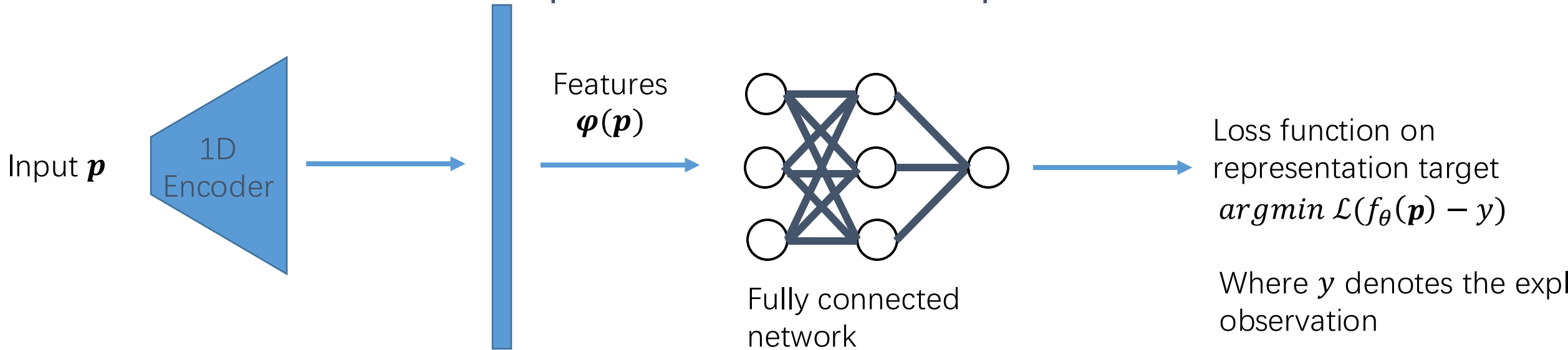


$$(x, y, z, \theta, \phi) \rightarrow \mathbf{F}_{\Theta} \rightarrow (RGB\sigma)$$



上海科技大学
ShanghaiTech University

Structure of implicit neural representation



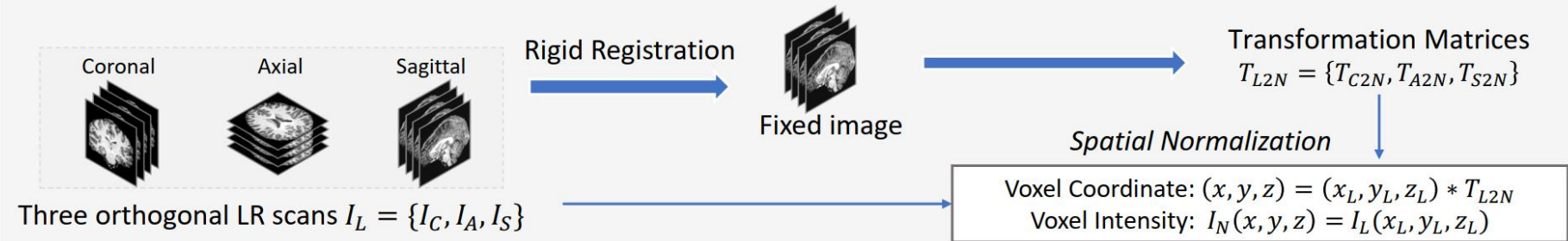
$$\boldsymbol{\varphi}(\mathbf{p}) = [a_1 \cos(2\pi \mathbf{b}_1^T \mathbf{p}), a_1 \sin(2\pi \mathbf{b}_1^T \mathbf{p}), \dots, a_m \cos(2\pi \mathbf{b}_m^T \mathbf{p}), a_m \sin(2\pi \mathbf{b}_m^T \mathbf{p})]^T$$

- Images: $f: \mathbb{R}^2 \rightarrow \mathbb{R}^3, f(x, y, z) = (R, G, B)$.
- Videos: $f: \mathbb{R}^3 \rightarrow \mathbb{R}^3, f(x, y, t) = (R, G, B)$.
- Shapes: $f: \mathbb{R}^3 \rightarrow \mathbb{R}^1, f(x, y, z) = 0 \text{ or } 1 \text{ as occupancy}$.

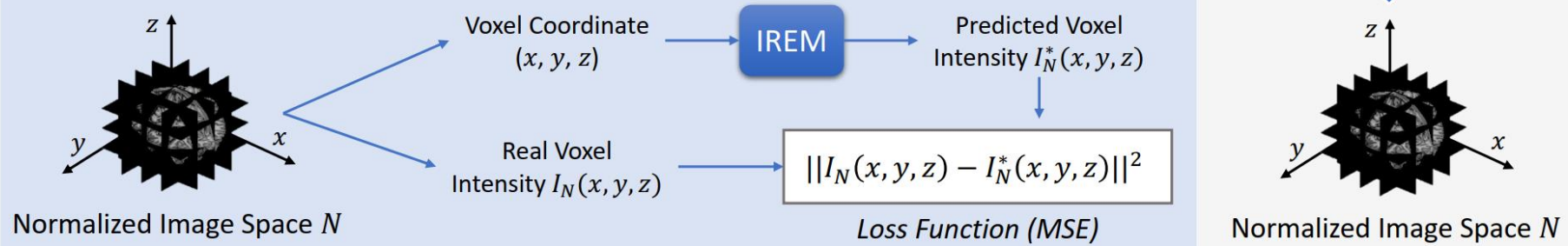


IREM

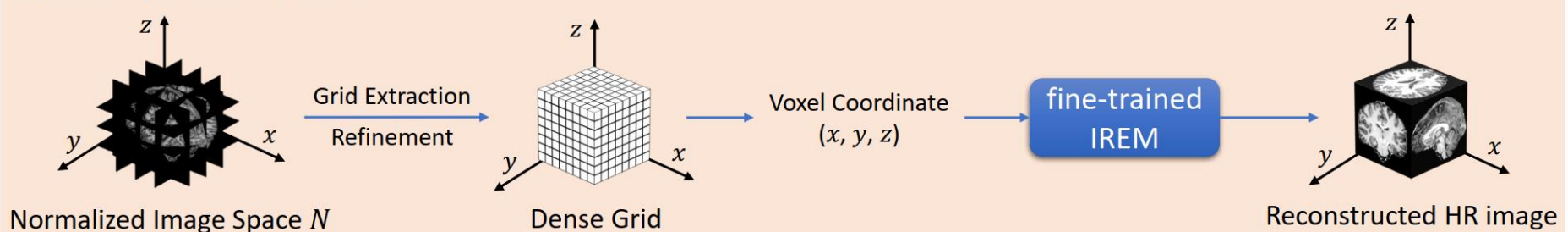
(a) Image Spatial Normalization



(b) Model Optimization



(c) HR Image Reconstruction

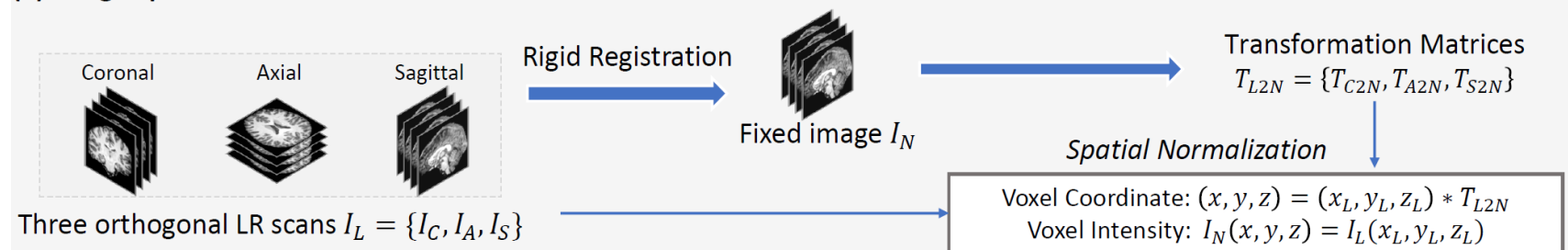


IREM-磁共振影像3D超分辨率重建

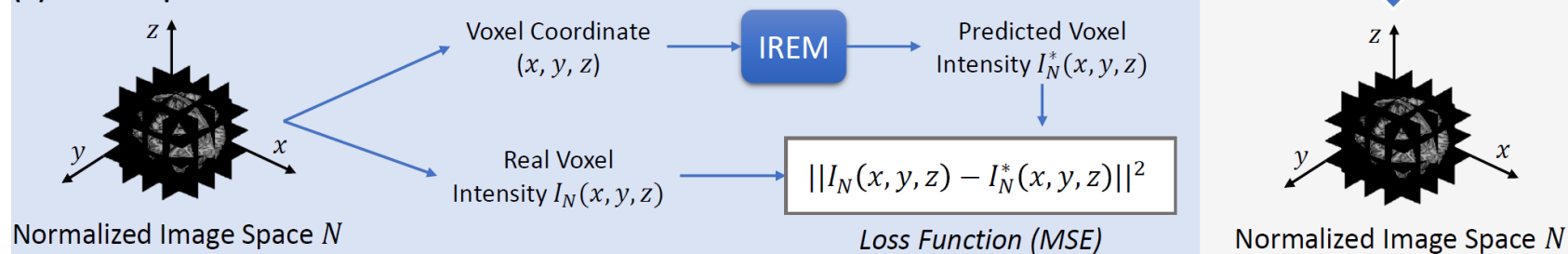
IREM: High-Resolution Magnetic Resonance (MR) Image Reconstruction via Implicit Neural Representation.

Architecture of Model Optimization

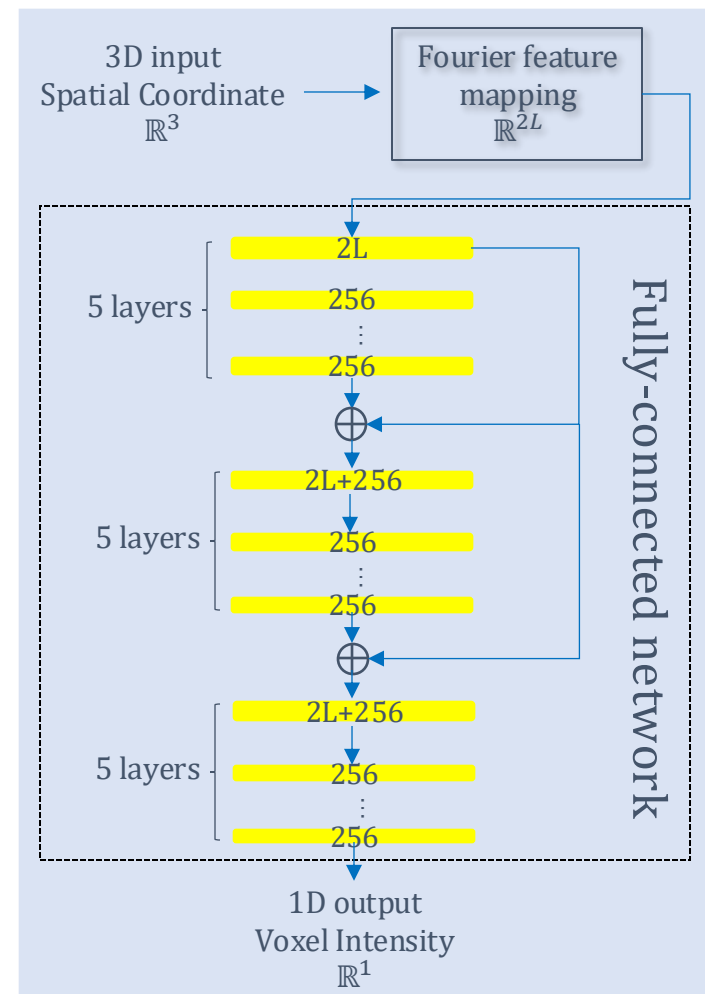
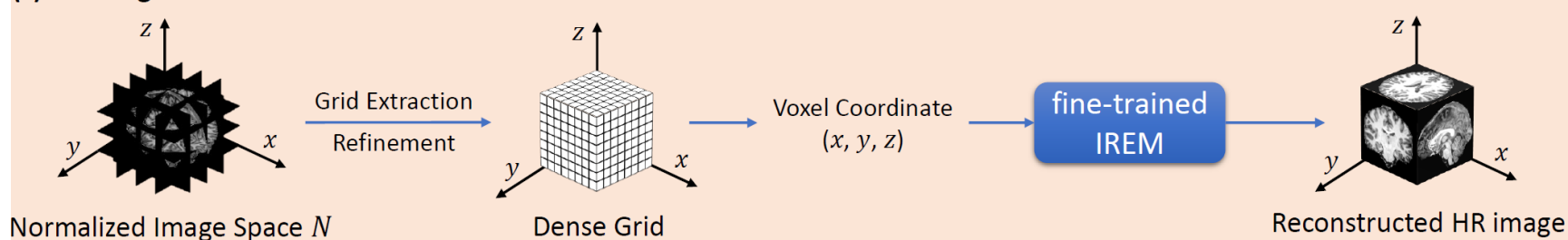
(a) Image Spatial Normalization An overview of the proposed IREM approach.



(b) Model Optimization

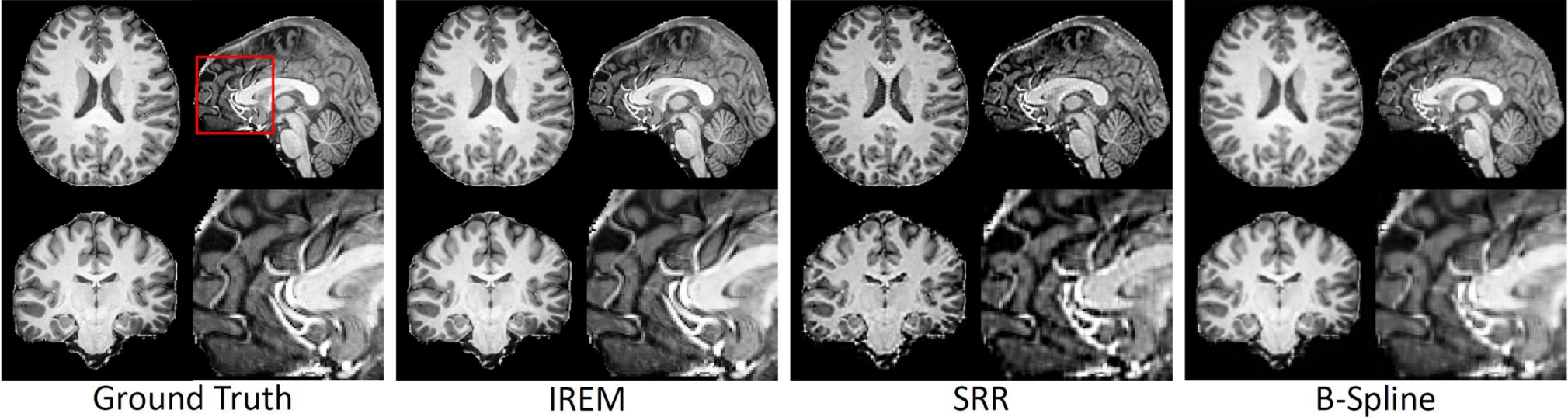


(c) HR Image Reconstruction

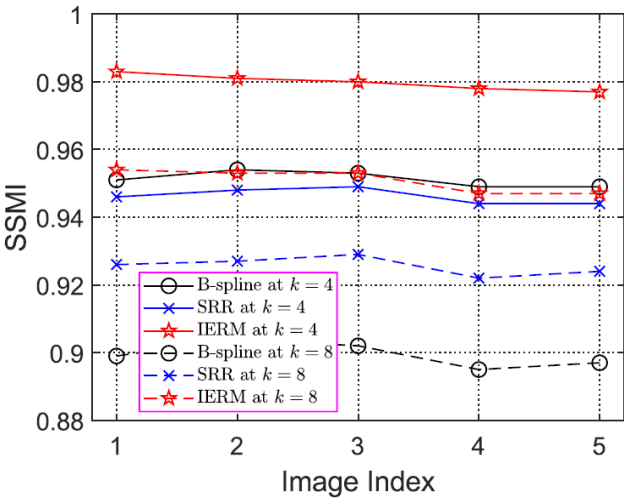
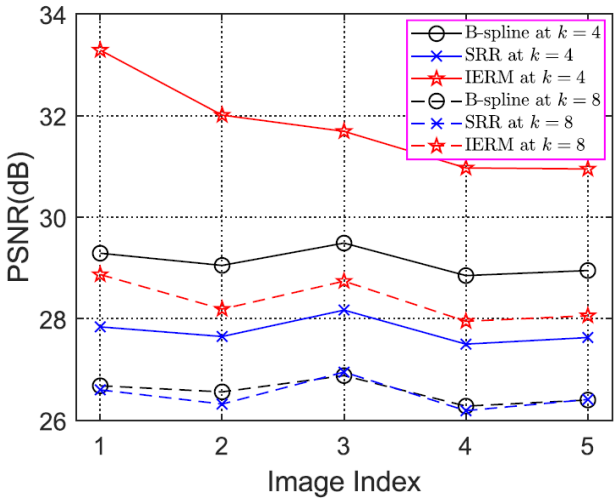


IREM-准确性验证

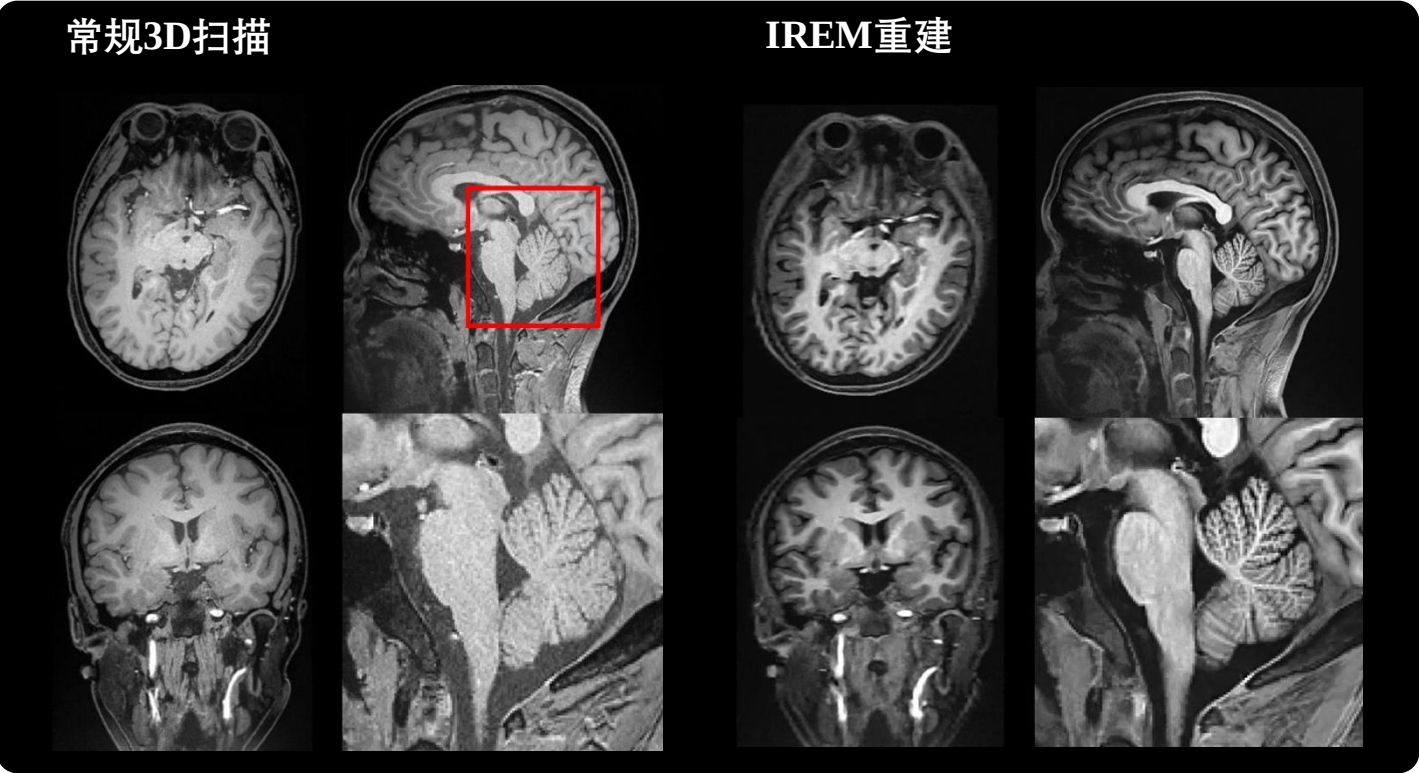
(a) On dataset #A (sub1)



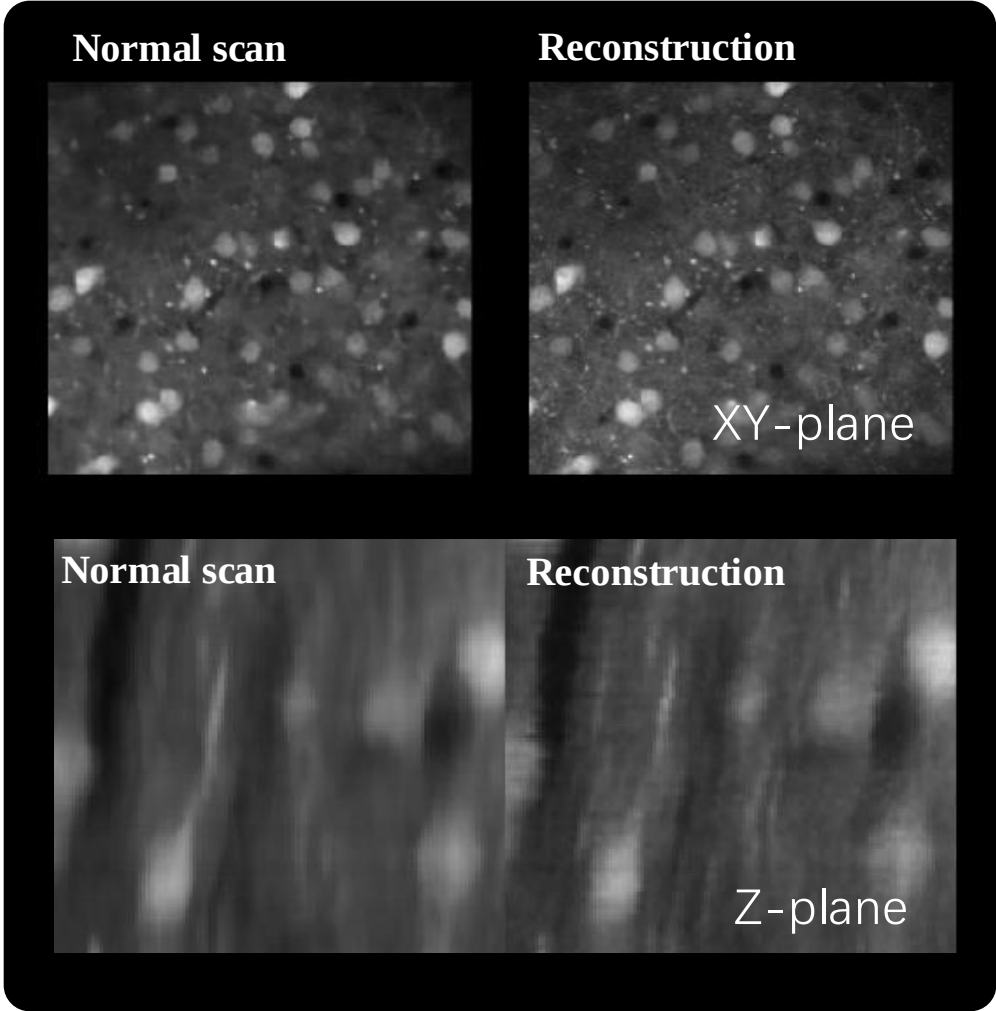
(b) On dataset #B (sub1)



IREM-实用性验证



HR reference	254	368×345	0.7×0.7×0.7	30.9 mins
axial scan	80	320×320	0.7×0.7×2.8	10.3 mins
coronal scan	80	320×320	0.7×2.8×0.7	10.3 mins
sagittal scan	64	320×320	2.8×0.7×0.7	8.2 mins



双光子显微镜超分辨重建

LIIF

- Arbitrary Scale Super Resolution for Arbitrary Images

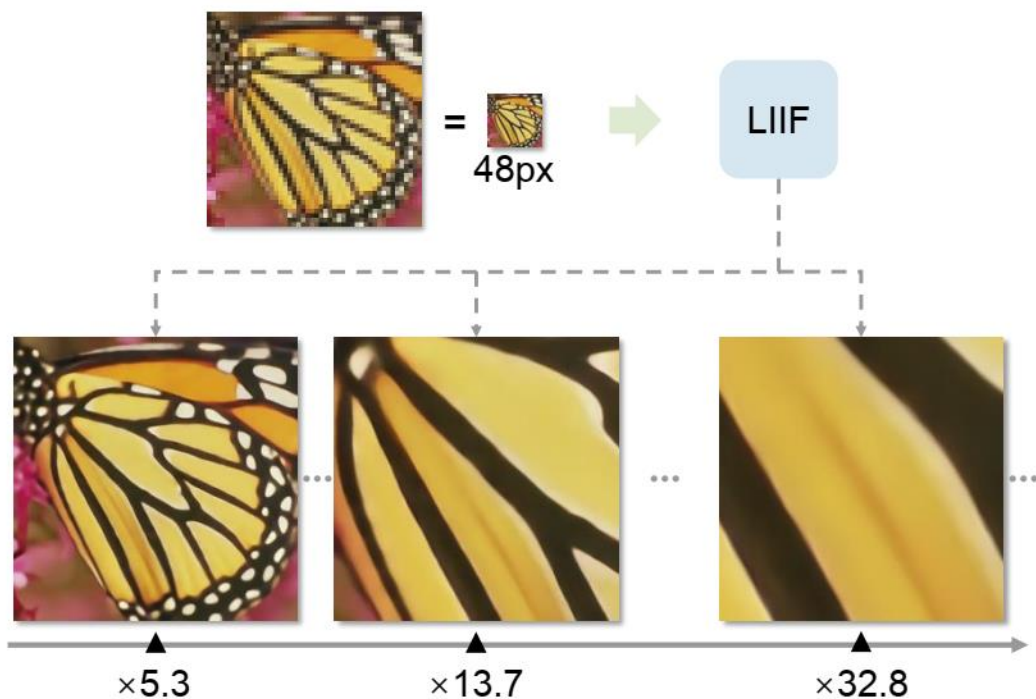


Figure 1: Local Implicit Image Function (LIIF) represents an image in continuous domain, which can be presented in arbitrary high resolution.

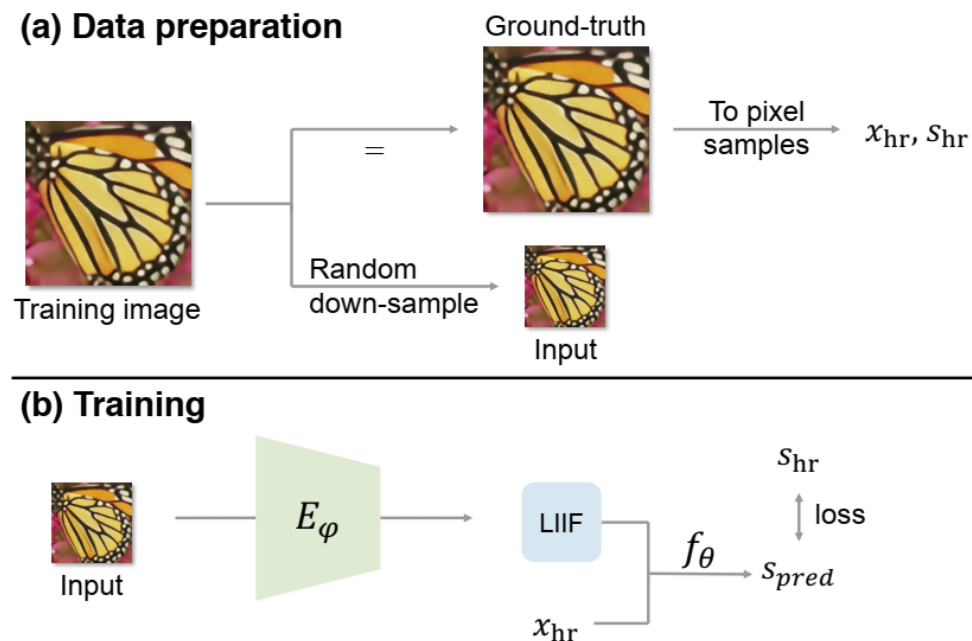


Figure 4: **Learning to generate continuous representation for pixel-based images.** An encoder is jointly trained with the LIIF representation in a self-supervised super-resolution task, in order to encourage the LIIF representation to maintain high fidelity in higher resolution.

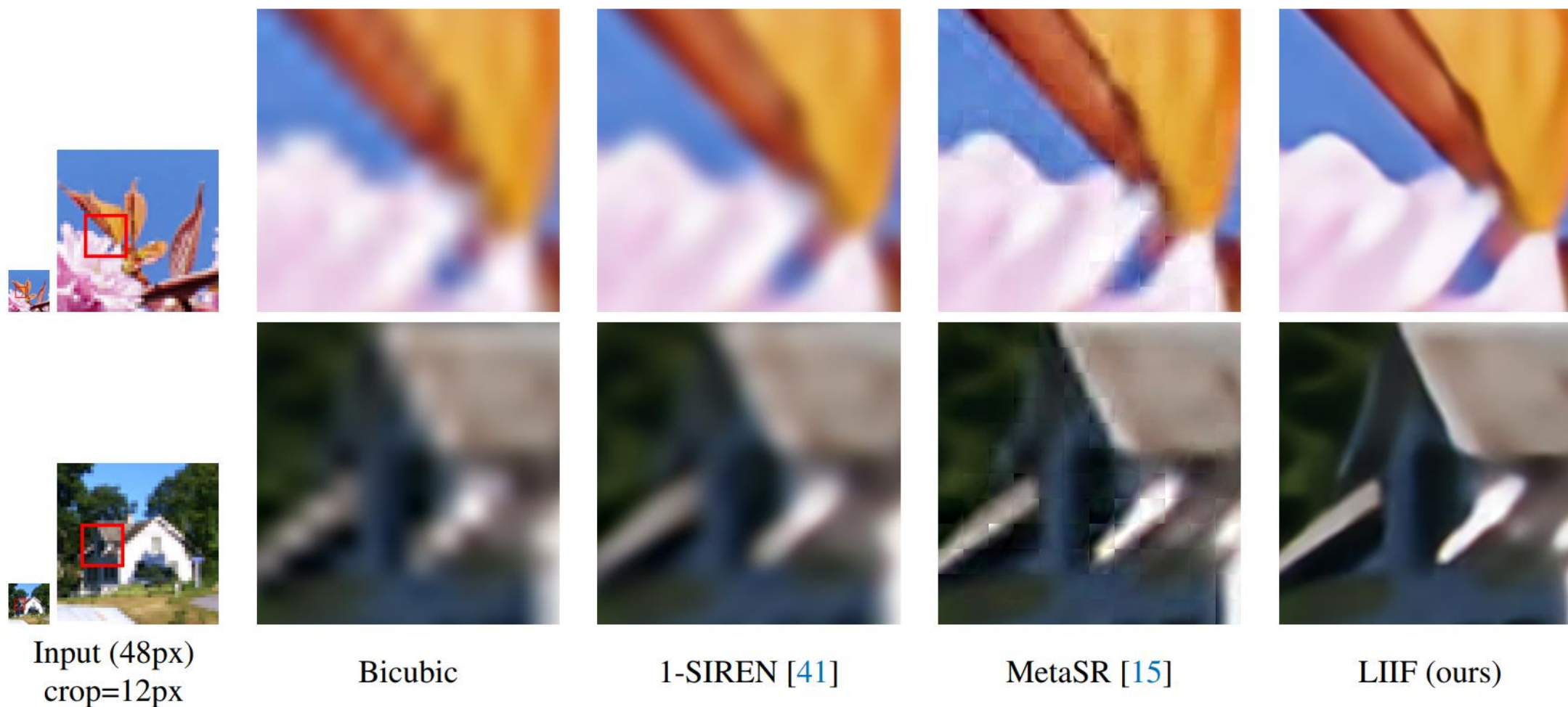


Figure 5: **Qualitative comparison of learning continuous representation.** The input is a 48×48 patch from images in DIV2K validation set, a red box indicates the crop area for demonstration ($\times 30$). 1-SIREN refers to fitting an independent implicit function for the input image. MetaSR and LIIF are trained for continuous random scales in $\times 1 - \times 4$ and tested for $\times 30$ for evaluating the generalization to arbitrary high precision of the continuous representation.

Deep Learning in Blind Super-Resolution

- [1] SRMD: Learning a Single Convolutional Super-Resolution Network for Multiple Degradations (CVPR 2018)
- [2] Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data (ICCV workshop 2021)

Contents

1. Introduction to blind SR
 - Image degradation model
 - Classical SR
 - What is blind SR? & Why study blind SR?
2. Two DL-based solutions for blind SR
 - SR based on estimated kernel (SRMD)
 - Degradation process simulation (Real-ESRGAN)
 - ~~Unpaired Supervised Learning~~



Image degradation model

- General degradation model for image acquisition



- Formally,

$$y = \mathbf{B}\mathbf{D}x + \mathbf{n}$$

- $\mathbf{D}$ is blur matrix
- $\mathbf{B}$ is down-sample matrix
- $\mathbf{n}$ is noise matrix

Image degradation model

- Degradation model for three image restorations

$$y = \mathbf{D}\mathbf{B}x + \mathbf{n}$$

- If both the $\mathbf{D}$ and $\mathbf{B}$ are identity matrices.



Real Image x

$$y = x + \mathbf{n}$$



Observation y

?

Image Denoise

Image degradation model

- Degradation model for three image restorations

$$y = \mathbf{D}\mathbf{B}x + \mathbf{n}$$

- If only $\mathbf{D}$ is identity matrix.



Real Image x

$$y = \mathbf{B}x + \mathbf{n}$$



Observation y

?

Image Deblur

Image degradation model

- Degradation model for three image restorations

$$y = \mathbf{D}\mathbf{B}x + \mathbf{n}$$

- If both the $\mathbf{D}$ and $\mathbf{B}$ are NOT identity matrices.



Real Image x

$$y = \mathbf{D}\mathbf{B}x + \mathbf{n}$$



Observation y

?

Image SR

Classical SR

➤ Two steps

1. Building simulation dataset from collected HR images.
2. Training SR models by the simulation dataset



HR Images x

$$y = \mathbf{D}\mathbf{B}x + \mathbf{n}$$



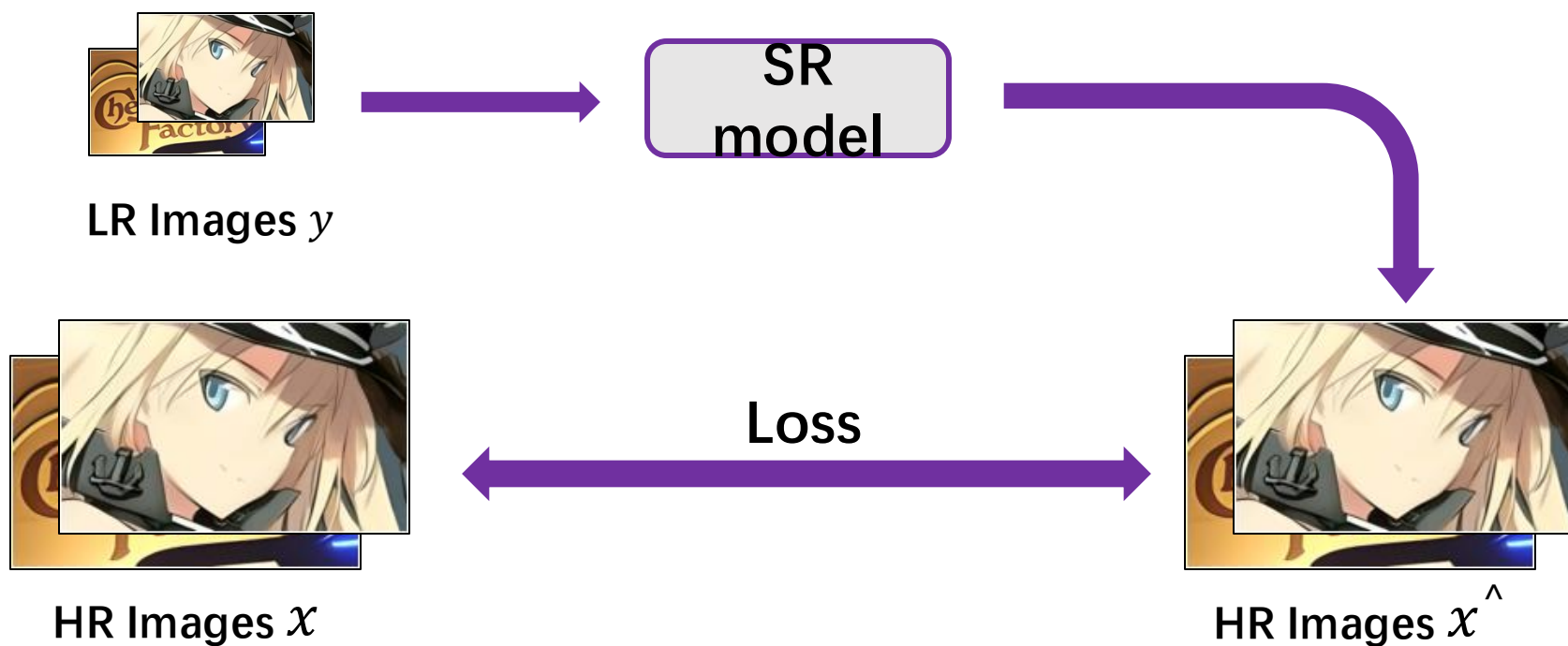
LR Images y

- $\mathbf{D}$ is cubic down-sample
- $\mathbf{B}$ is Gaussian blur
- $\mathbf{n}$ is additive Gaussian noise

Classical SR

➤ Two steps

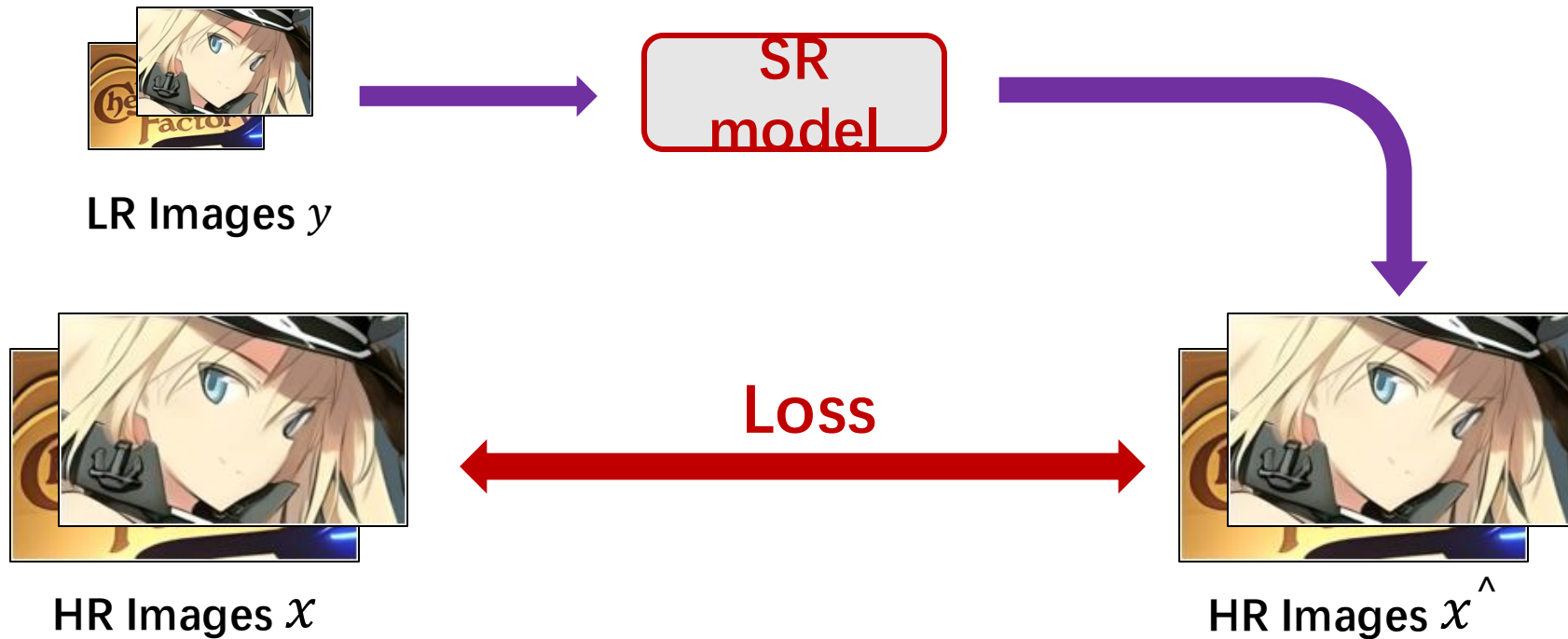
1. Building simulation dataset from collected HR images.
2. Training SR models by the simulation dataset



Classical SR

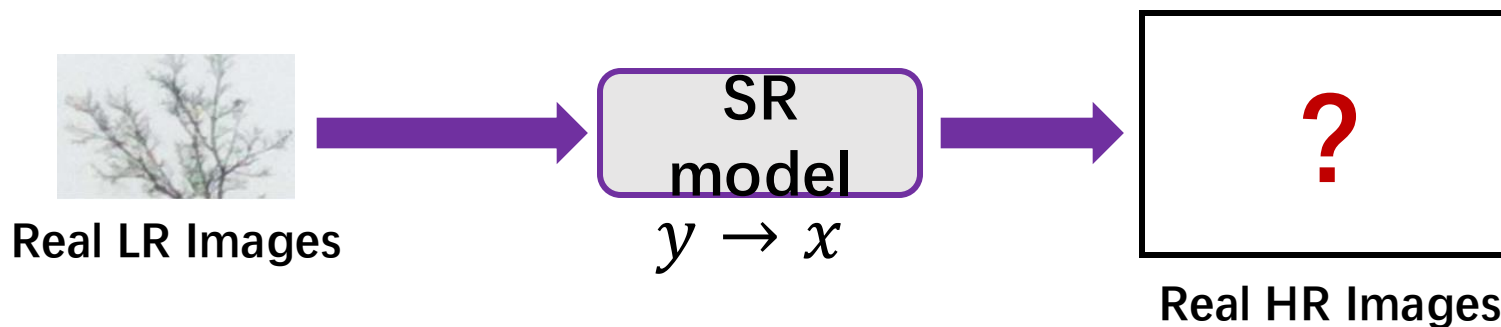
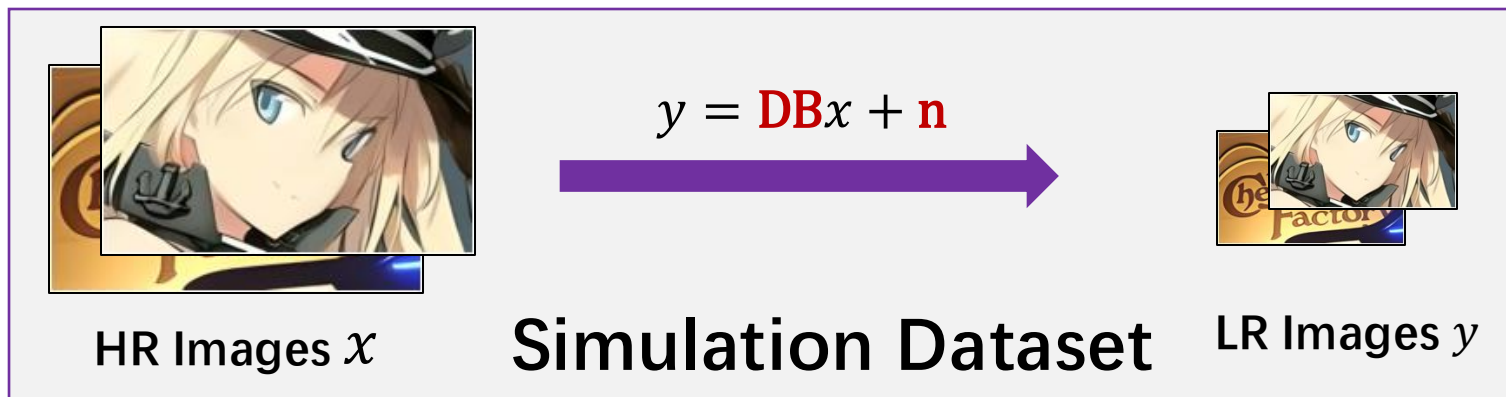
➤ Two steps

1. Building simulation dataset from collected HR images.
2. Training SR models by the simulation dataset



Classical SR

- Degradation model bias
 - The true degradation does **NOT** follow the simulation degradation.

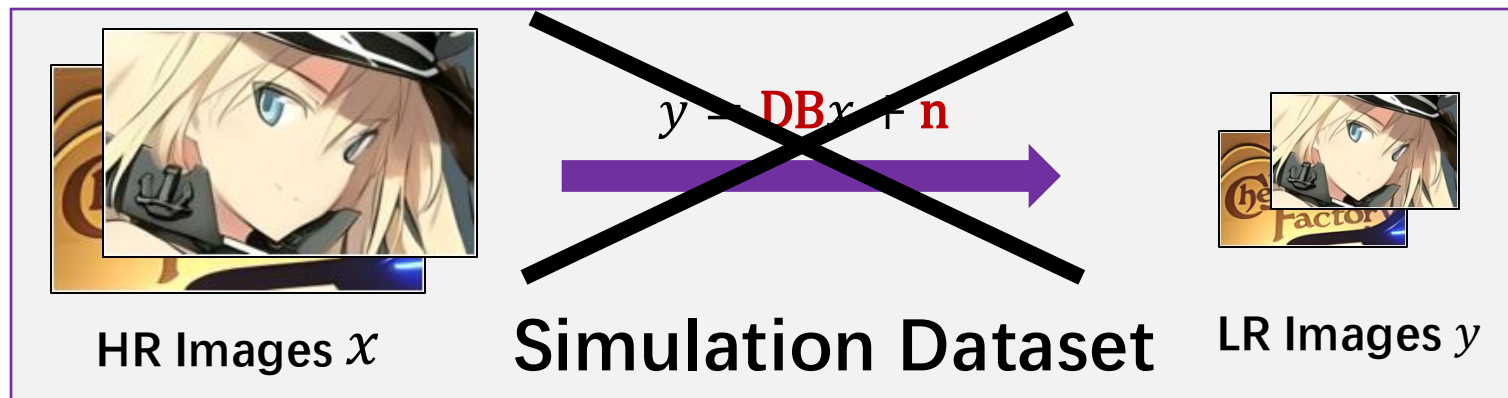


What is blind SR? & Why study blind SR?

- Degradation model is **unknown**

$$y = \mathbf{D}\mathbf{B}x + \mathbf{n}$$

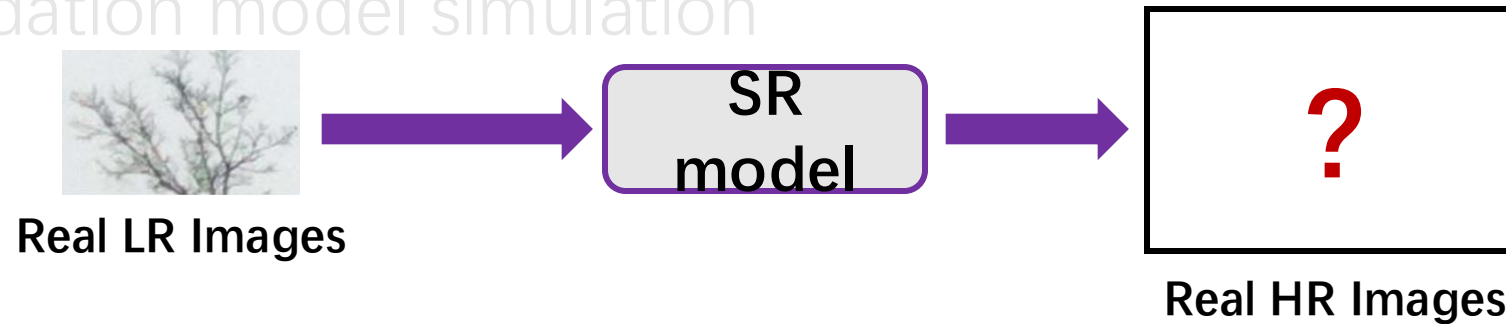
- i.e., both the **D**, **B**, and **n** are unknown



- SR without **fixed** degradation assumption
 - It can avoid degradation model bias

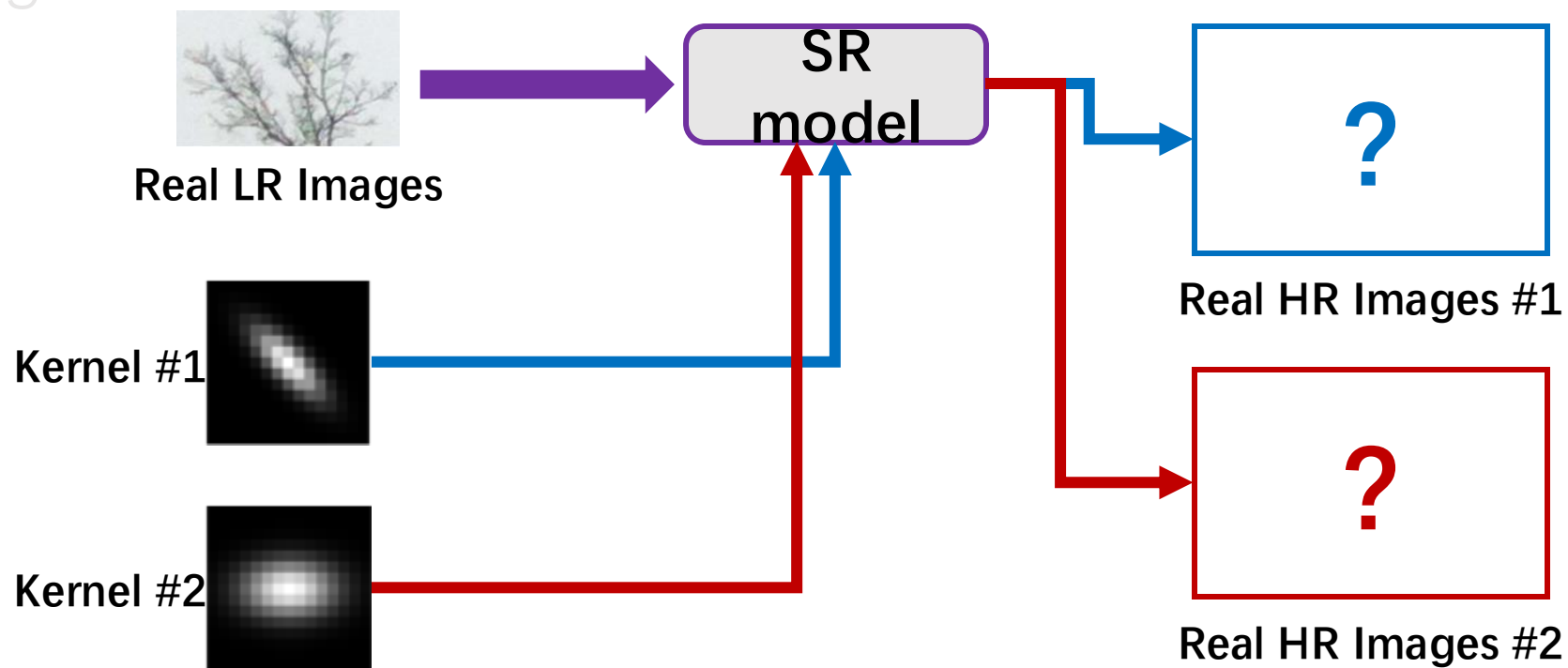
Two DL-based solutions for blind SR

1. SR based on estimated kernel
2. Degradation model simulation



Two DL-based solutions for blind SR

1. SR based on estimated kernel
2. Degradation model simulation



SRMD

➤ **Paper Title**

- Learning a Single Convolutional Super-Resolution Network for Multiple Degradations

➤ **Author list**

- Kai Zhang, Wangmeng Zuo, and, Lei Zhang

➤ **Affiliations**

- Harbin Institute of Technology
- The Hong Kong Polytechnic University
- DAMO Academy, Alibaba Group

➤ **Conference**

- CVPR 2018

SRMD

➤ Key point

- The LR image and kernel have **different** dimensions

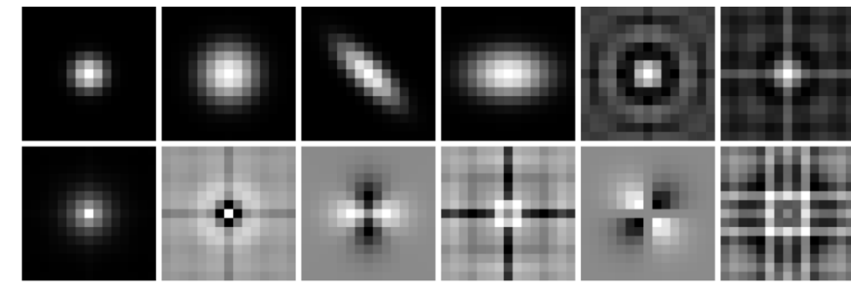
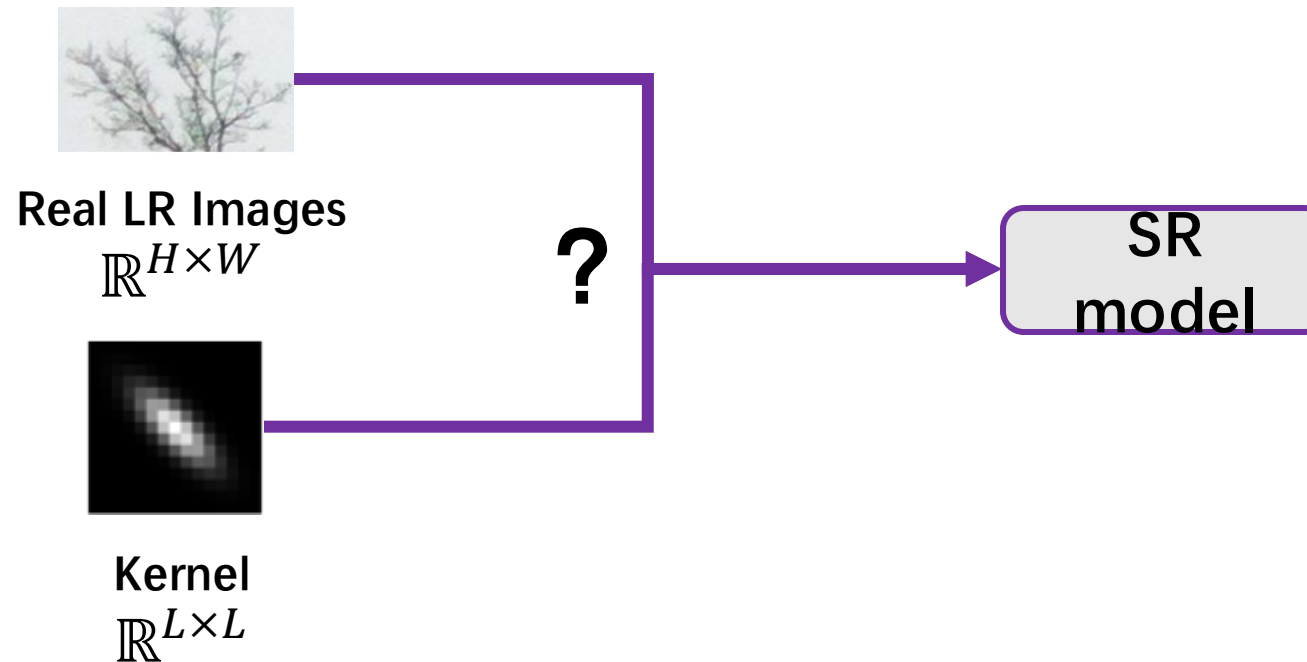
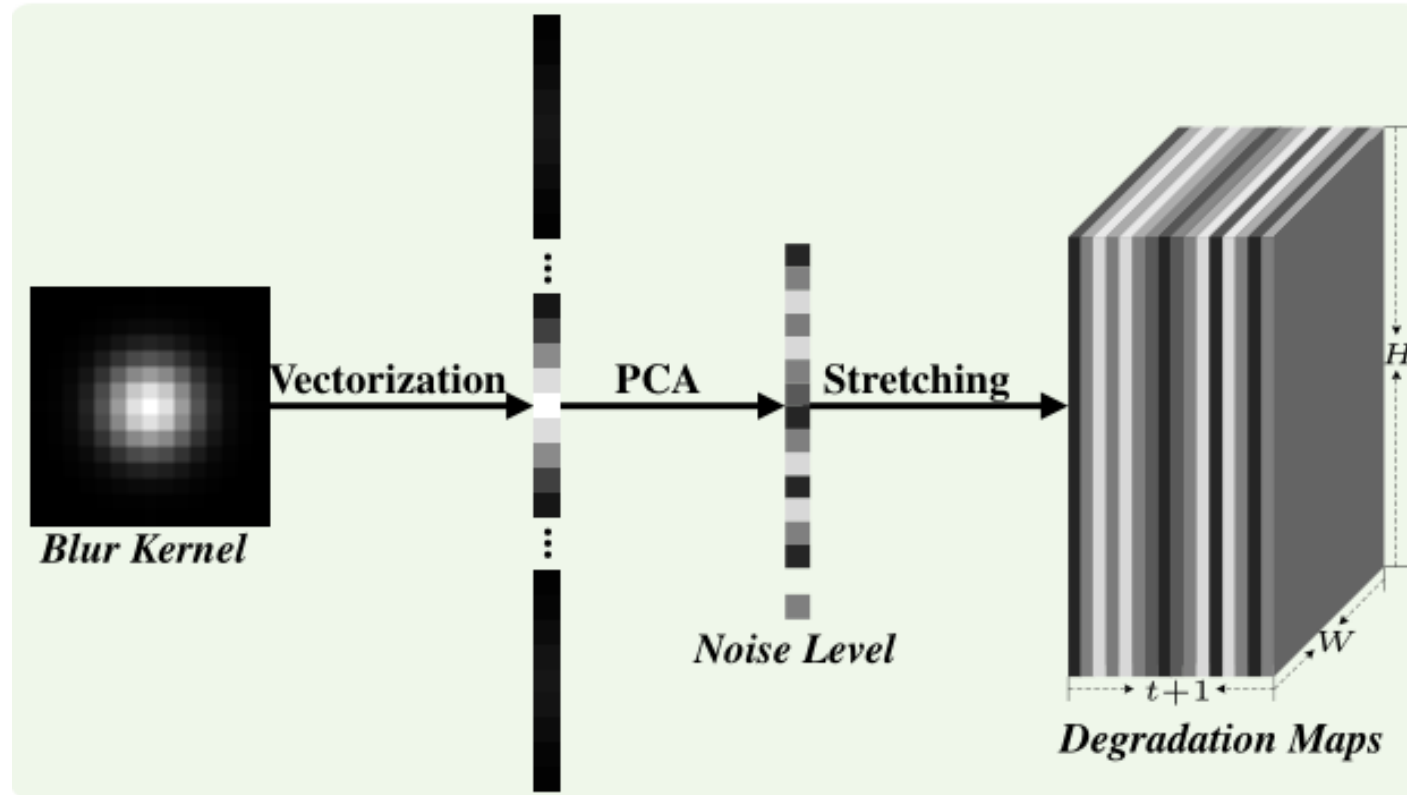


Figure 4. Visualization of six typical blur kernels (**first row**) of isotropic Gaussian (first two), anisotropic Gaussian (middle two) and estimated ones for direct downsampler (last two) for scale factor 3 and PCA eigenvectors (**second row**) for the first six largest eigenvalues.

SRMD: Method

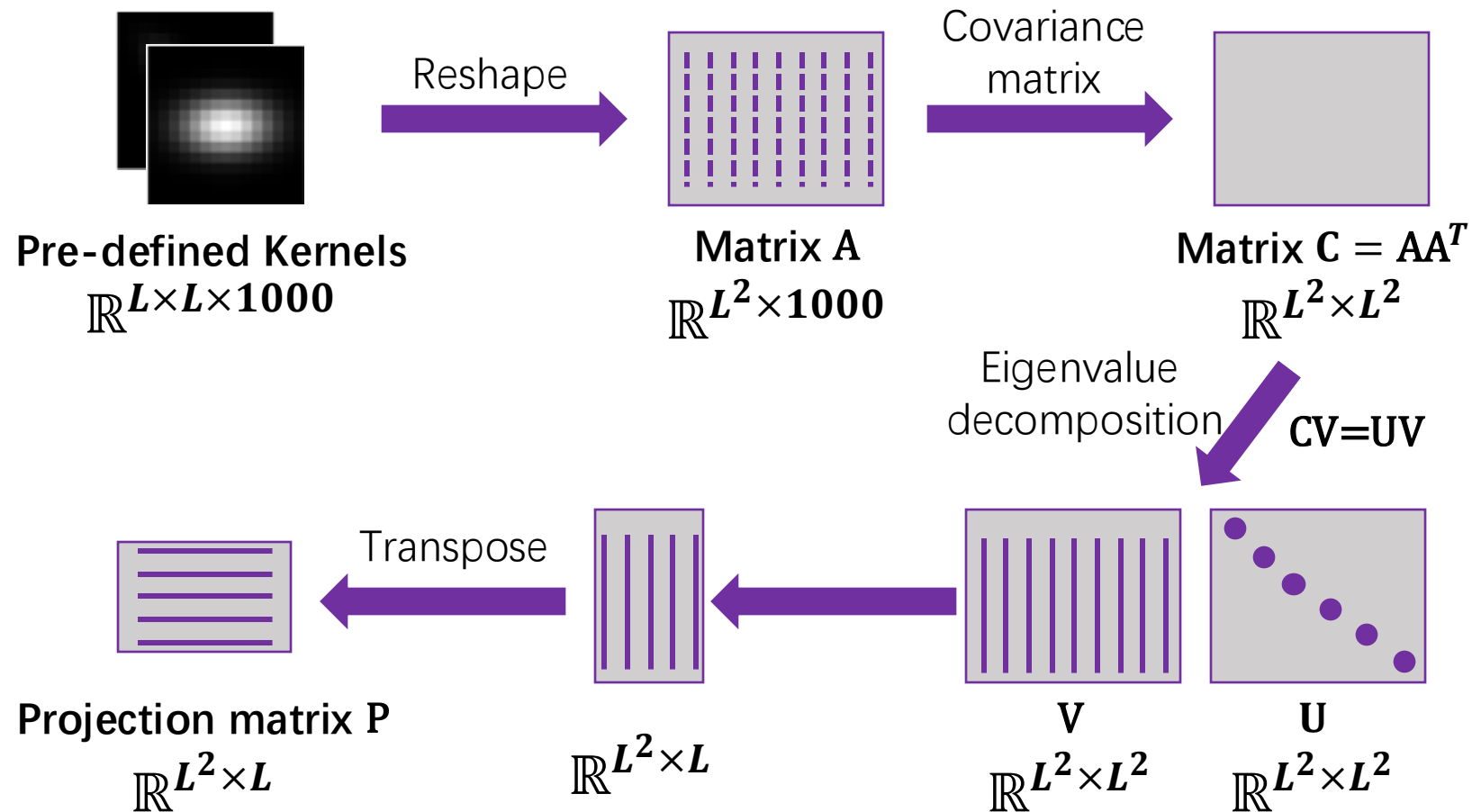
➤ Dimensionality Stretching



$$\mathbb{R}^{L \times L} \longrightarrow \mathbb{R}^{L^2} \longrightarrow \mathbb{R}^t \longrightarrow \mathbb{R}^{H \times W \times (t+1)}$$

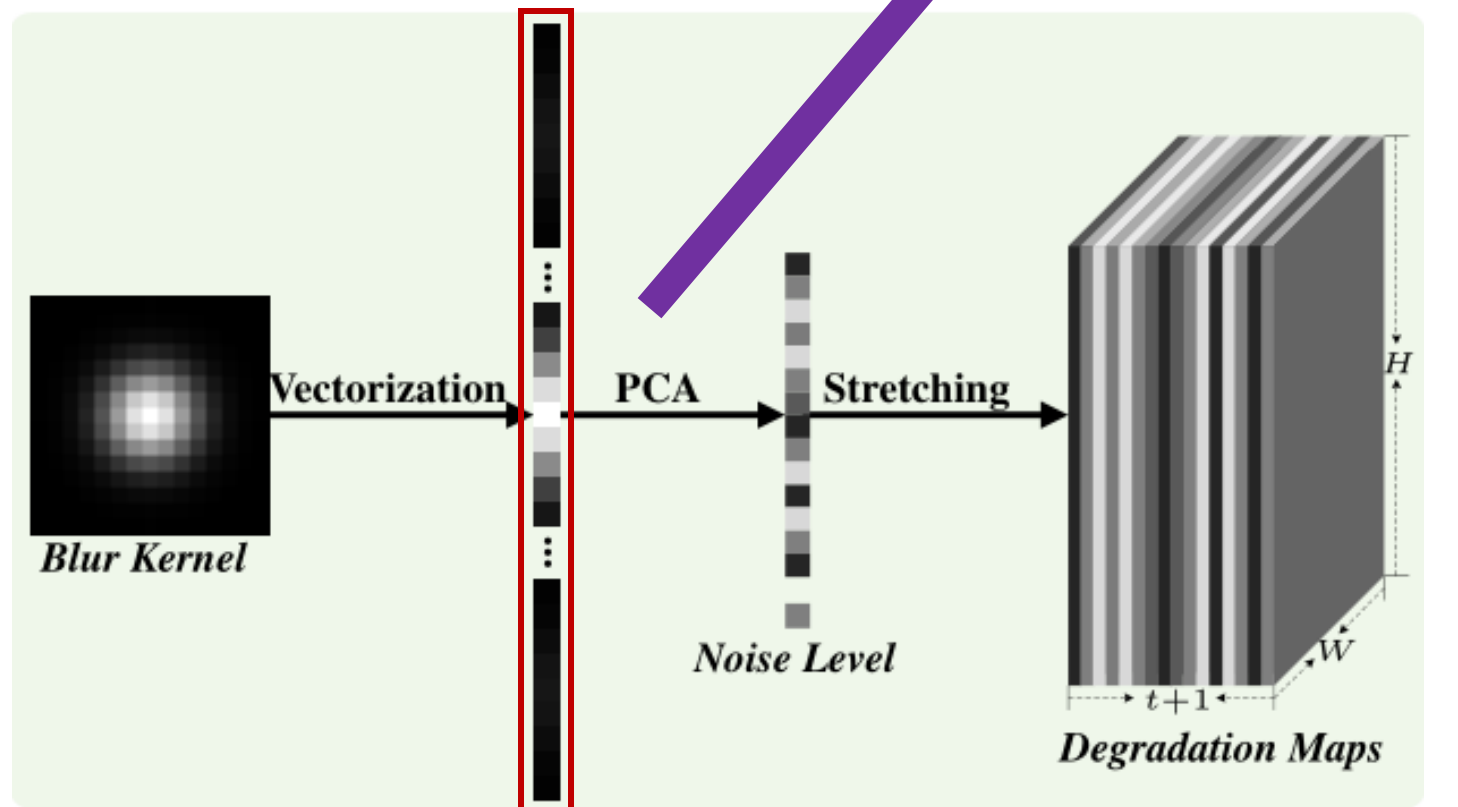
SRMD: Method

- How to generate PCA projection matrix ?



SRMD: Method

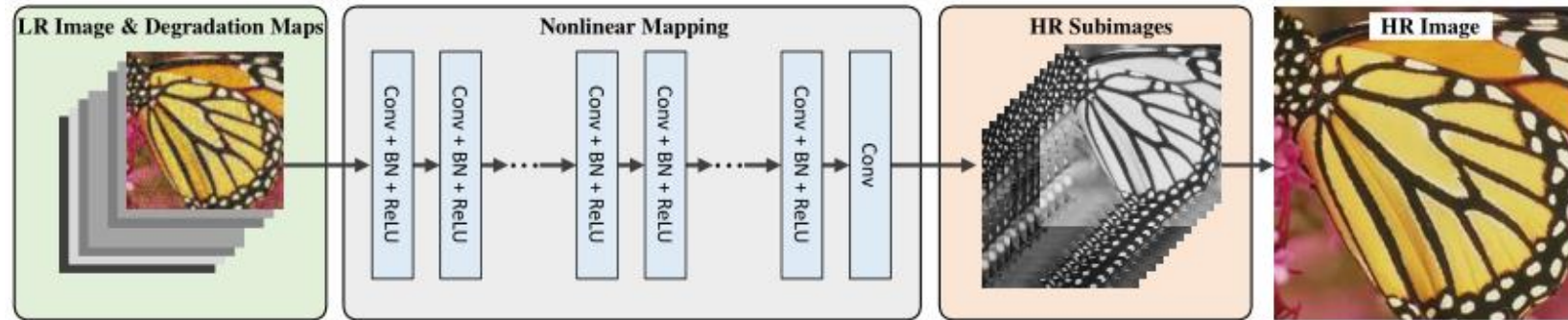
➤ Dimensionality Stretching



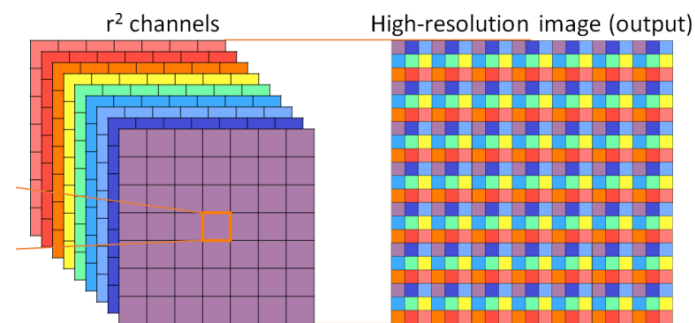
$$\mathbb{R}^{L \times L} \rightarrow \mathbb{R}^{L^2 \times 1} \rightarrow \mathbb{R}^{L+1} \rightarrow \mathbb{R}^{H \times W \times (L+1)}$$

SRMD: Method

➤ Overview of the SRMD



$$\mathbb{R}^{H \times W \times (L+1+C)} \longrightarrow \mathbb{R}^{H \times W \times s^2 C} \longrightarrow \mathbb{R}^{sH \times sW \times C}$$



Sub-pixel Conv Layer

SRMD: Experiments on bicubic kernel

Table 1. Average PSNR and SSIM results for bicubic degradation on datasets Set5 [3], Set14 [54], BSD100 [33] and Urban100 [19]. The best two results are highlighted in red and blue colors, respectively.

Dataset	Scale Factor	Bicubic	SRCNN [9]	VDSR [24]	SRResNet [29]	DRRN [44]	LapSRN [27]	SRMD	SRMDNF
		PSNR / SSIM							
Set5	×2	33.64 / 0.929	36.62 / 0.953	37.56 / 0.959	–	37.66 / 0.959	37.52 / 0.959	37.53 / 0.959	37.79 / 0.960
	×3	30.39 / 0.868	32.74 / 0.908	33.67 / 0.922	–	33.93 / 0.923	33.82 / 0.922	33.86 / 0.923	34.12 / 0.925
	×4	28.42 / 0.810	30.48 / 0.863	31.35 / 0.885	32.05 / 0.891	31.58 / 0.886	31.54 / 0.885	31.59 / 0.887	31.96 / 0.893
Set14	×2	30.22 / 0.868	32.42 / 0.906	33.02 / 0.913	–	33.19 / 0.913	33.08 / 0.913	33.12 / 0.914	33.32 / 0.915
	×3	27.53 / 0.774	29.27 / 0.821	29.77 / 0.832	–	29.94 / 0.834	29.89 / 0.834	29.84 / 0.833	30.04 / 0.837
	×4	25.99 / 0.702	27.48 / 0.751	27.99 / 0.766	28.49 / 0.780	28.18 / 0.770	28.19 / 0.772	28.15 / 0.772	28.35 / 0.777
BSD100	×2	29.55 / 0.843	31.34 / 0.887	31.89 / 0.896	–	32.01 / 0.897	31.80 / 0.895	31.90 / 0.896	32.05 / 0.898
	×3	27.20 / 0.738	28.40 / 0.786	28.82 / 0.798	–	28.91 / 0.799	28.82 / 0.798	28.87 / 0.799	28.97 / 0.803
	×4	25.96 / 0.667	26.90 / 0.710	27.28 / 0.726	27.58 / 0.735	27.35 / 0.726	27.32 / 0.727	27.34 / 0.728	27.49 / 0.734
Urban100	×2	26.66 / 0.841	29.53 / 0.897	30.76 / 0.914	–	31.02 / 0.916	30.82 / 0.915	30.89 / 0.916	31.33 / 0.920
	×3	24.46 / 0.737	26.25 / 0.801	27.13 / 0.828	–	27.38 / 0.833	27.07 / 0.828	27.27 / 0.833	27.57 / 0.840
	×4	23.14 / 0.657	24.52 / 0.722	25.17 / 0.753	–	25.35 / 0.758	25.21 / 0.756	25.34 / 0.761	25.68 / 0.773

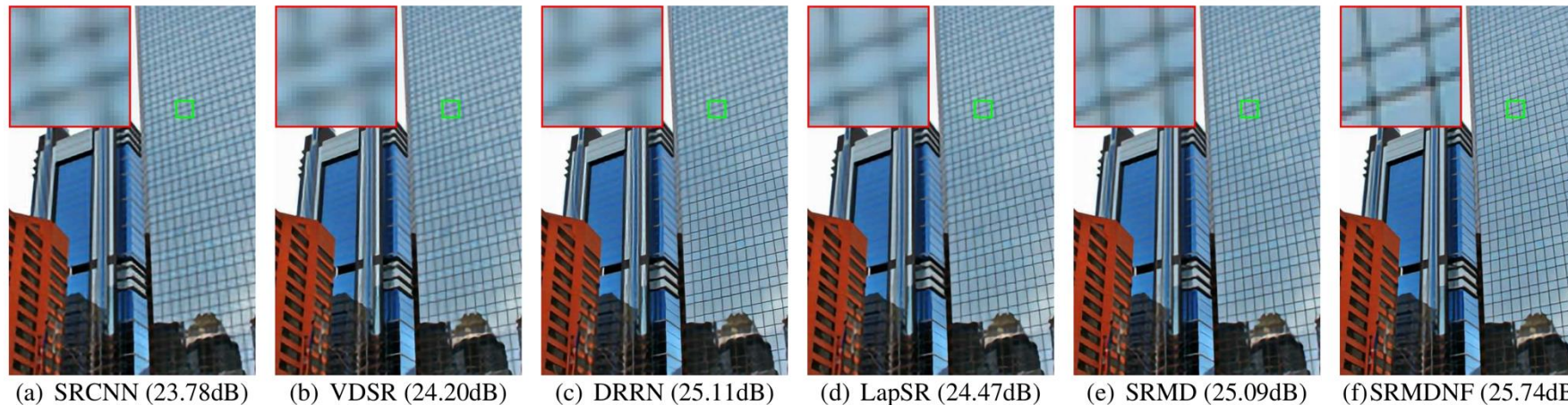


Figure 5. SISR performance comparison of different methods with scale factor 4 on image “*Img_099*” from Urban100.

SRMD: Experiments on general kernel

Table 2. Average PSNR and SSIM results of different methods with different degradations on Set5. The best results are highlighted in red color. The results highlighted in gray color indicate unfair comparison due to mismatched degradation assumption.

Degradation Settings			VDSR [24]	NCSR [11]	IRCNN [57]	DnCNN [56]+SRMDNF	SRMD	SRMDNF
Kernel Width	Down-sampler	Noise Level	PSNR ($\times 2/\times 3/\times 4$)					
0.2	Bicubic	0	37.56/33.67/31.35	– /23.82/–	37.43/33.39/31.02	–	37.53/33.86/31.59	37.79/34.12/31.96
0.2	Bicubic	15	26.02/25.40/24.70	–	32.60/30.08/28.35	32.47/30.07/28.31	32.76/30.43/28.79	–
0.2	Bicubic	50	16.02/15.72/15.46	–	28.20/26.25/24.95	28.20/26.27/24.93	28.51/26.48/25.18	–
1.3	Bicubic	0	30.57/30.24/29.72	– /21.81/–	36.01/33.33/31.01	–	37.04/33.77/31.56	37.45/34.16/31.99
1.3	Bicubic	15	24.82/24.70/24.30	–	29.96/28.68/27.71	27.68/28.78/27.71	30.98/29.43/28.21	–
1.3	Bicubic	50	15.89/15.68/15.43	–	26.69/25.20/24.42	24.35/25.19/24.39	27.43/25.82/24.77	–
2.6	Bicubic	0	26.37/26.31/26.28	– /21.46/–	32.07/31.09/30.06	–	33.24/32.59/31.20	34.12/33.02/31.77
2.6	Bicubic	15	23.09/23.07/22.98	–	26.44/25.67/24.36	– /21.33/23.85	28.48/27.55/26.82	–
2.6	Bicubic	50	15.58/15.43/15.23	–	22.98/22.16/21.43	– /19.03/21.15	25.85/24.75/23.98	–
1.6	Direct	0	– /30.54/ –	– /33.02/ –	– /33.38/ –	–	– /33.74/ –	– /34.01/ –

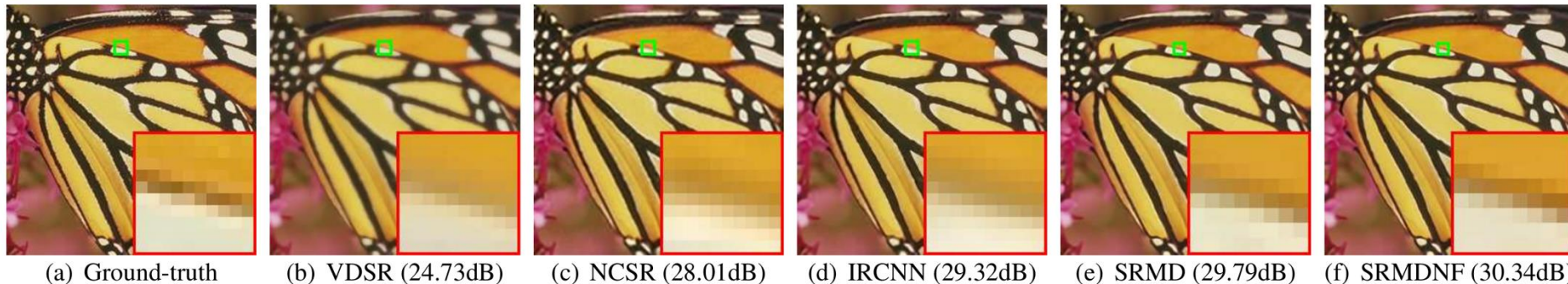
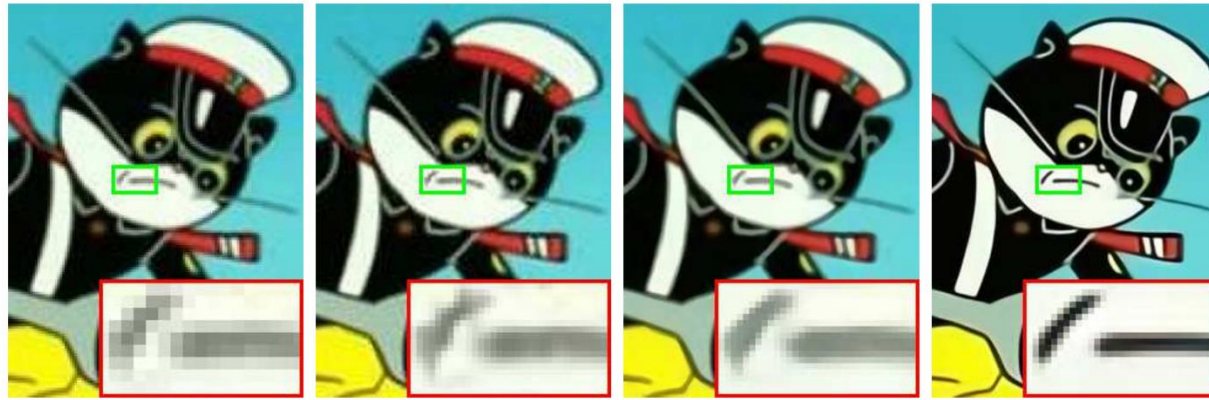


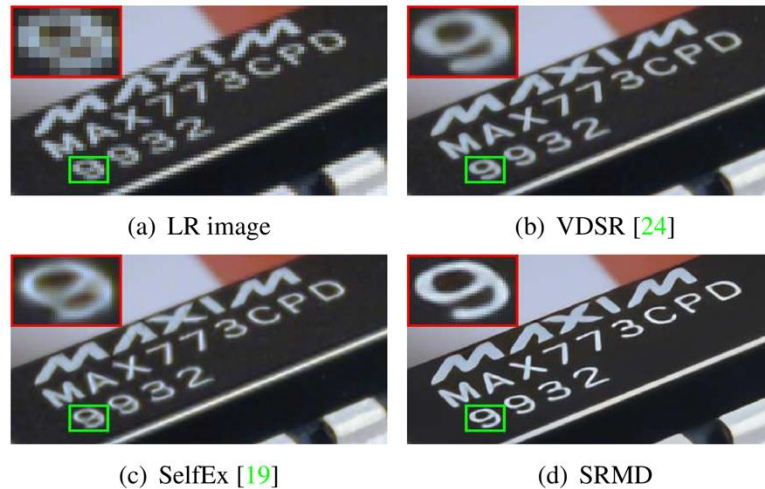
Figure 6. SISR performance comparison on image “Butterfly” from Set5. The degradation involves 7×7 Gaussian kernel with width 1.6 and direct downsampler with scale factor 3. Note that the comparison with VDSR is unfair because of degradation mismatch.

SRMD: Experiments on real image



(a) LR image (b) VDSR [24] (c) Waifu2x [49] (d) SRMD

Figure 8. SISR results on image “Cat” with scale factor 2.

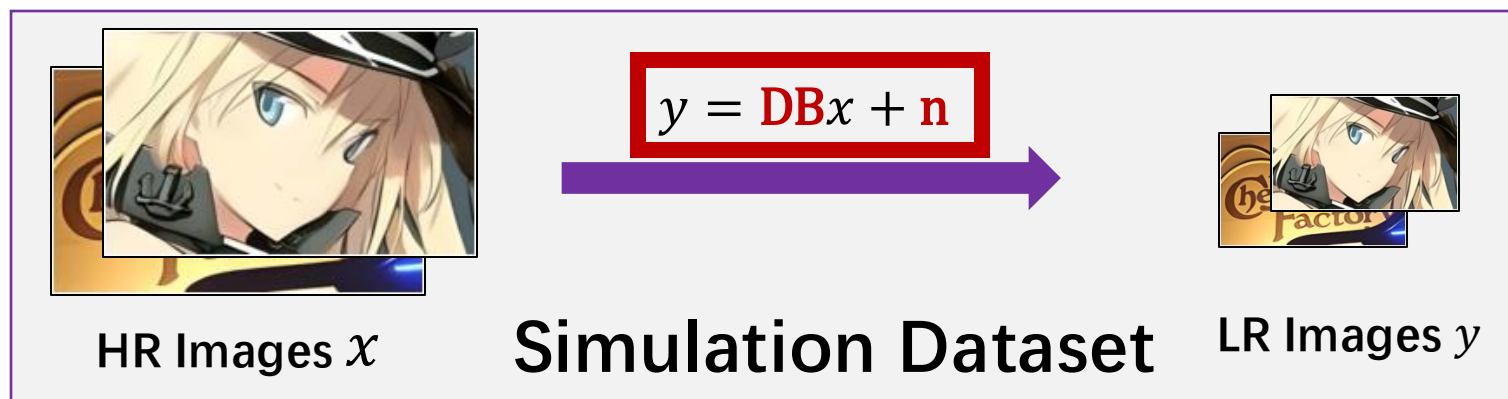


(a) LR image (b) VDSR [24] (c) SelfEx [19] (d) SRMD

Figure 9. SISR results on real image “Chip” with scale factor 4.

Two DL-based solutions for blind SR

1. SR based on estimated kernel
2. Degradation model simulation



- How to simulate complex degradation process ?

Real-ESRGAN: Method

➤ Degradation model in the Real-ESRGAN

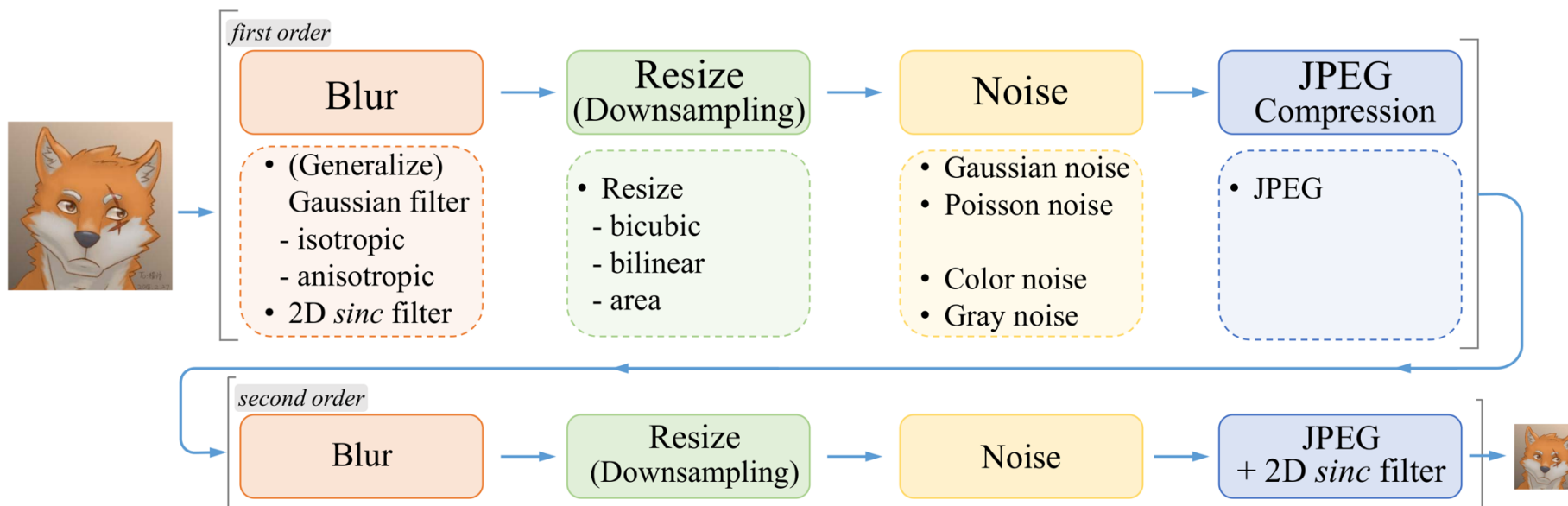


Figure 2: Overview of the pure synthetic data generation adopted in Real-ESRGAN. It utilizes a second-order degradation process to model more practical degradations, where each degradation process adopts the classical degradation model. The detailed choices for blur, resize, noise and JPEG compression are listed. We also employ *sinc* filter to synthesize common ringing and overshoot artifacts.

Real-ESRGAN: Method

➤ Overview of the Real-ESRGAN

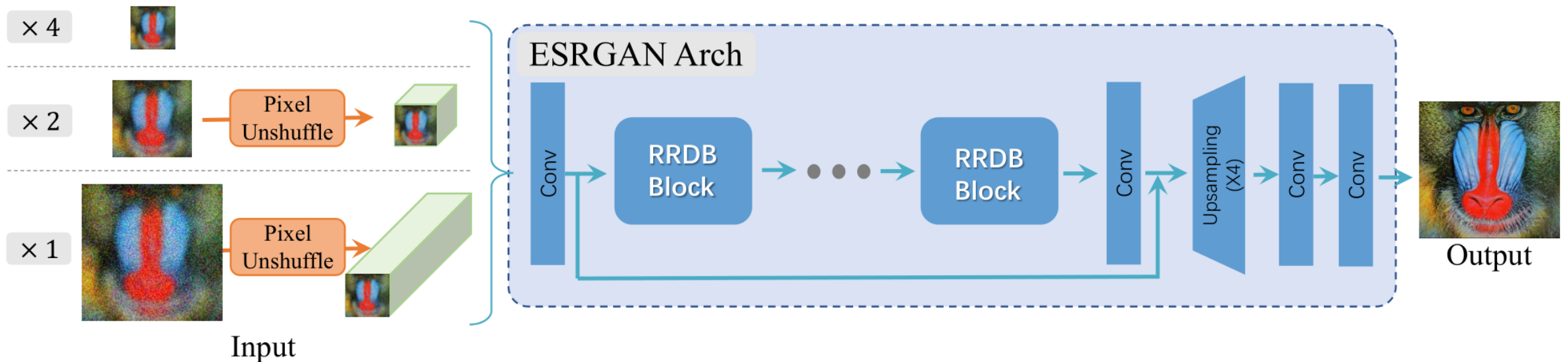


Figure 4: Real-ESRGAN adopts the same generator network as that in ESRGAN. For the scale factor of $\times 2$ and $\times 1$, it first employs a pixel-unshuffle operation to reduce spatial size and re-arrange information to the channel dimension.

Real-ESRGAN: Method

➤ Overview of the Real-ESRGAN

$D(x_r) = \sigma(C(\text{Real})) \rightarrow 1 \quad \text{Real?}$		$D_{Ra}(x_r, x_f) = \sigma(C(\text{Real}) - \mathbb{E}[C(\text{Fake})]) \rightarrow 1 \quad \text{More realistic than fake data?}$
$D(x_f) = \sigma(C(\text{Fake})) \rightarrow 0 \quad \text{Fake?}$	➔	$D_{Ra}(x_f, x_r) = \sigma(C(\text{Fake}) - \mathbb{E}[C(\text{Real})]) \rightarrow 0 \quad \text{Less realistic than real data?}$
a) Standard GAN		b) Relativistic GAN

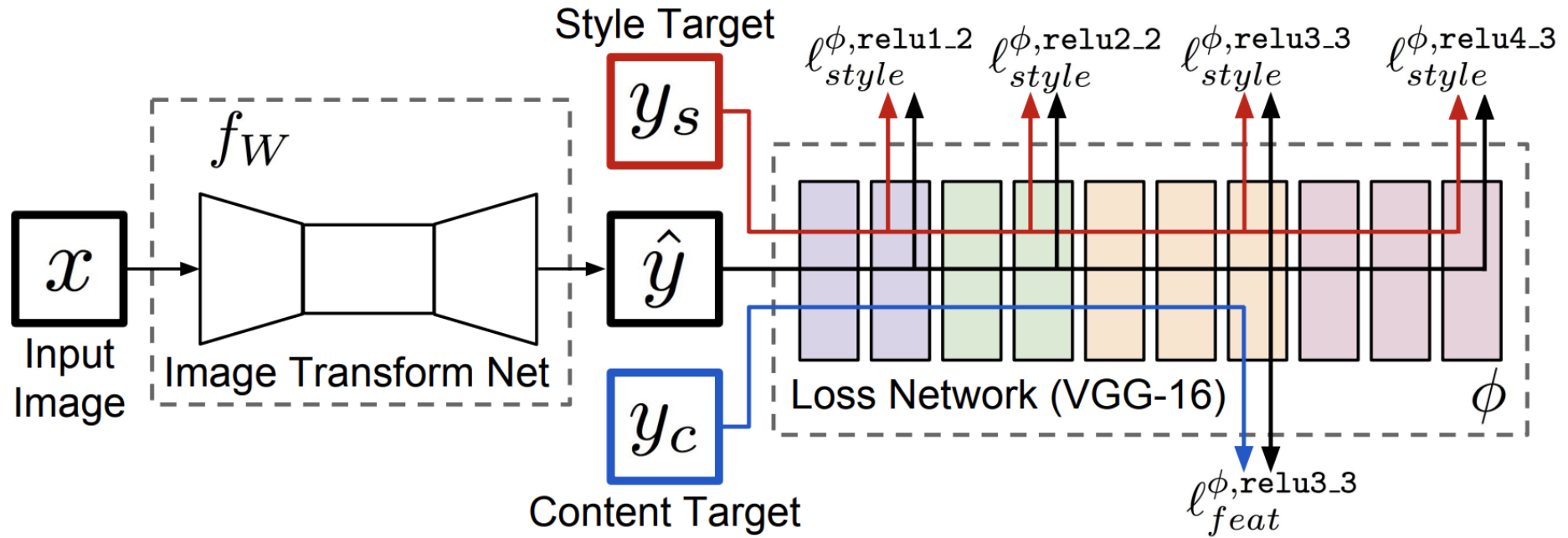
Generative Models: SRGAN & EnhanceNet & ESRGAN

- [1] Perceptual Losses for Real-Time Style Transfer and Super-Resolution (ECCV 2016)
- [2] Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network (CVPR 2017)

Perceptual Loss

- Feature Reconstruction Loss. Rather than encouraging the pixels of the output image $\hat{y} = f_W(x)$ to exactly match the pixels of the target image y , we instead encourage them to have similar feature representations as computed by the loss network ϕ .
- The networks discussed so far give equal importance to all spatial locations . In all our experiments ϕ is the 16-layer VGG network [46] pretrained on the ImageNet dataset.

Perceptual Loss



Feature Reconstruction Loss

$$\ell_{feat}^{\phi, j}(\hat{y}, y) = \frac{1}{C_j H_j W_j} \|\phi_j(\hat{y}) - \phi_j(y)\|_2^2$$

Style Reconstruction Loss.

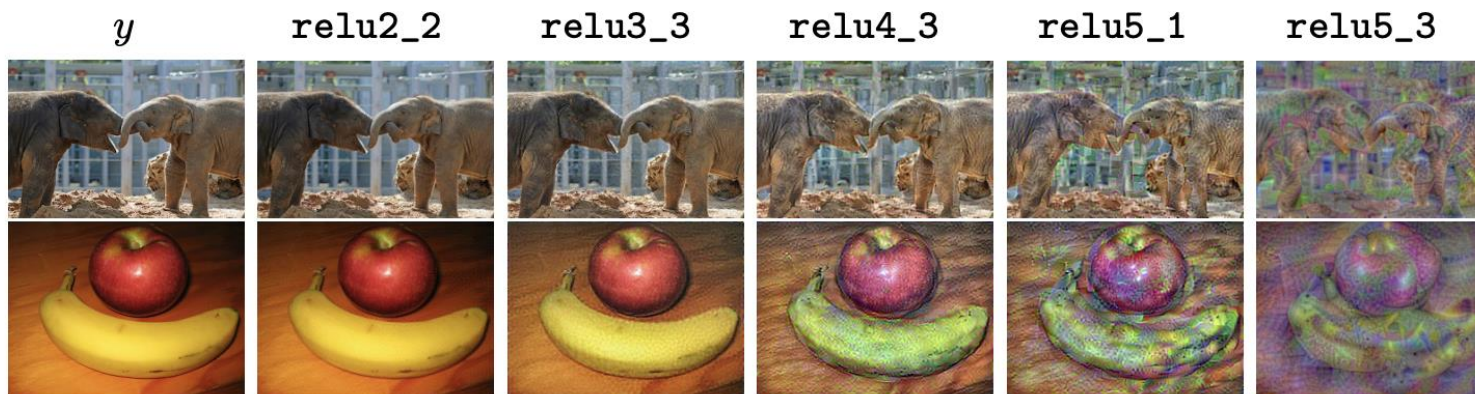
$$\ell_{style}^{\phi, j}(\hat{y}, y) = \|G_j^{\phi}(\hat{y}) - G_j^{\phi}(y)\|_F^2.$$

$$G_j^{\phi}(x)_{c, c'} = \frac{1}{C_j H_j W_j} \sum_{h=1}^{H_j} \sum_{w=1}^{W_j} \phi_j(x)_{h, w, c} \phi_j(x)_{h, w, c'}.$$

Perceptual Loss

Feature Reconstruction Loss

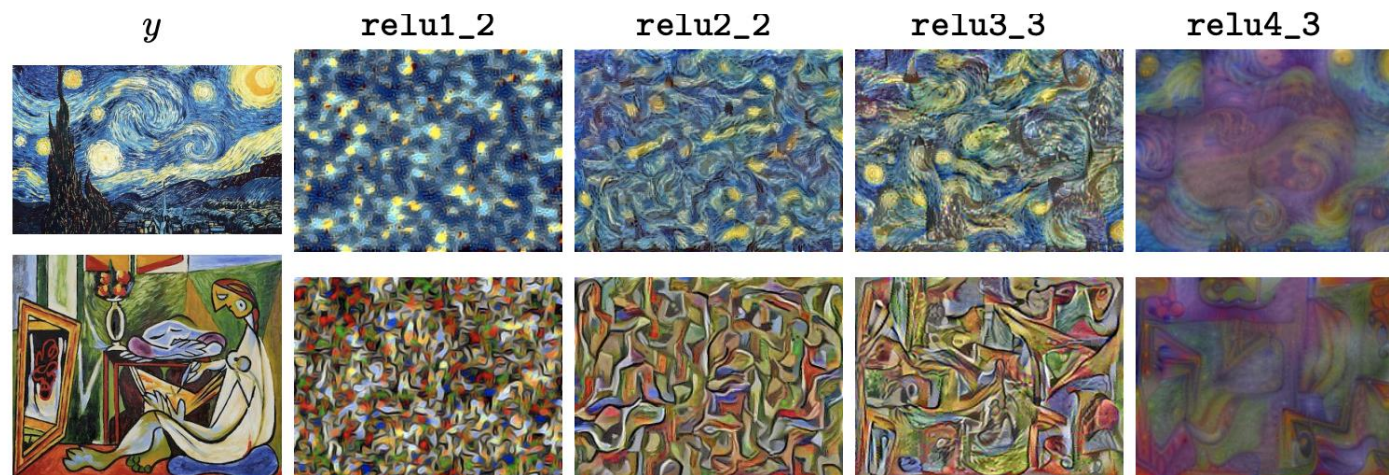
$$\ell_{feat}^{\phi,j}(\hat{y}, y) = \frac{1}{C_j H_j W_j} \|\phi_j(\hat{y}) - \phi_j(y)\|_2^2$$



Style Reconstruction Loss.

$$\ell_{style}^{\phi,j}(\hat{y}, y) = \|G_j^{\phi}(\hat{y}) - G_j^{\phi}(y)\|_F^2.$$

$$G_j^{\phi}(x) = \psi\psi^T / C_j H_j W_j.$$

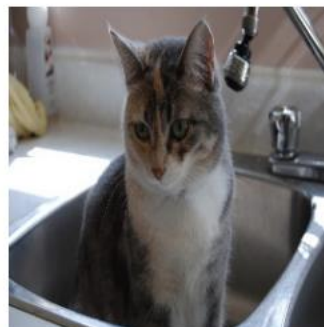


Perceptual Loss

Style
Sketch



Style
The Simpsons



Content

[10]

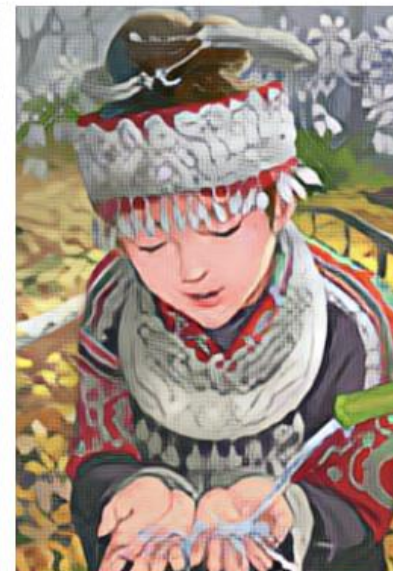
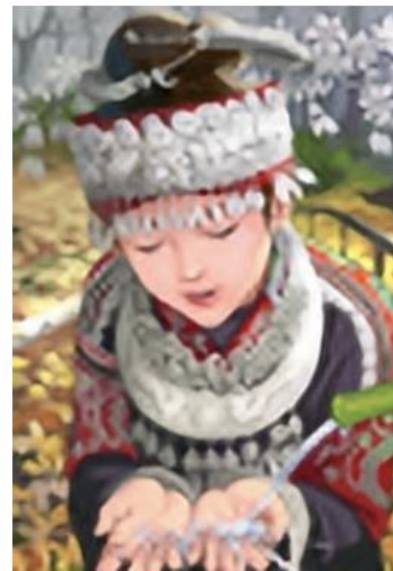
Ours

Content

[10]

Ours

Perceptual Loss



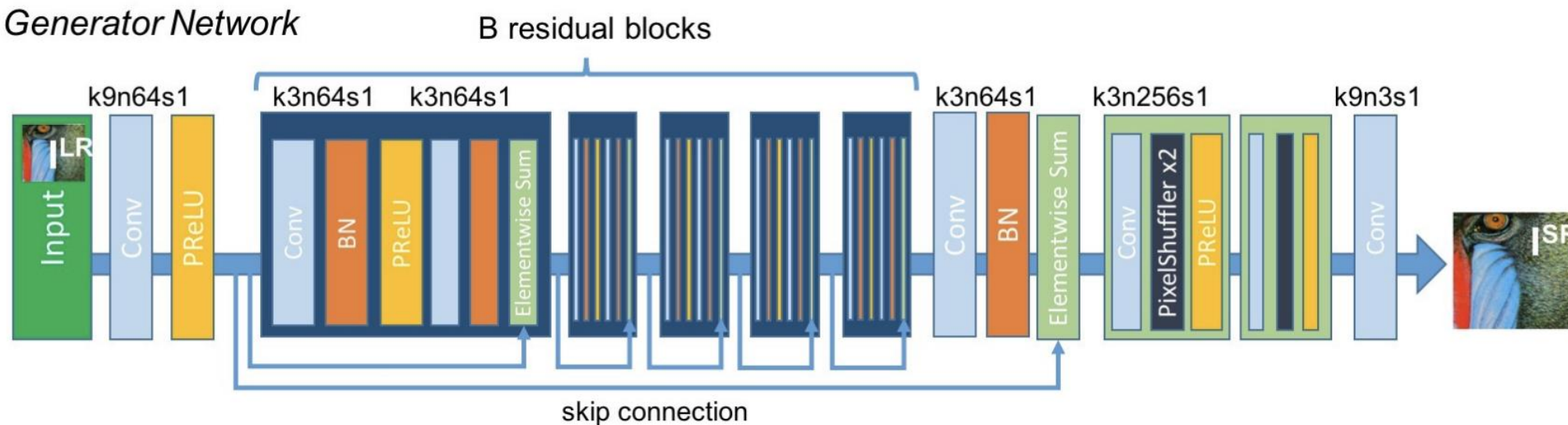
Ground Truth	Bicubic	Ours (ℓ_{pixel})	SRCNN [11]	Ours (ℓ_{feat})
This Image	21.69 / 0.5840	21.66 / 0.5881	22.53 / 0.6524	21.04 / 0.6116
Set14 mean	25.99 / 0.7301	25.75 / 0.6994	27.49 / 0.7503	24.99 / 0.6731
BSD100 mean	25.96 / 0.682	25.91 / 0.6680	26.90 / 0.7101	24.95 / 63.17

SRGAN

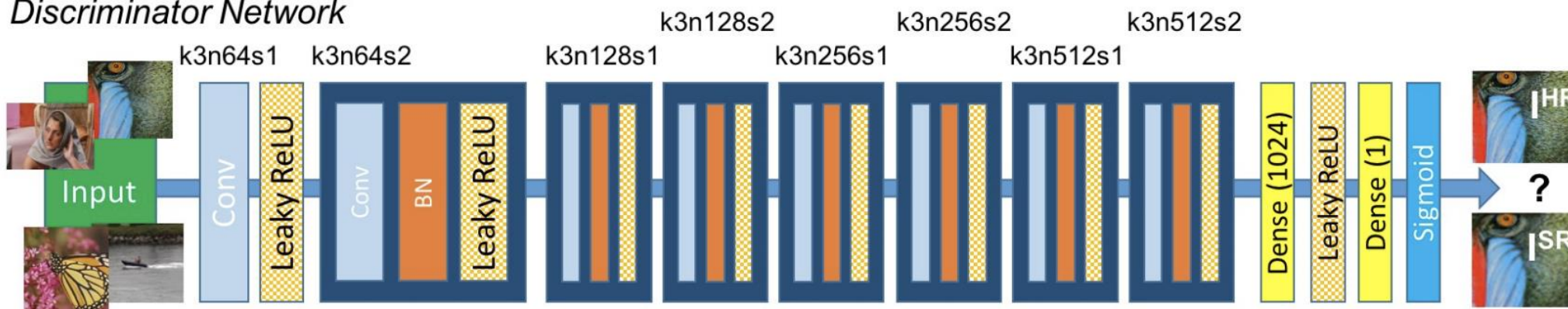
- It is the first framework capable of inferring photo-realistic natural images for $4\times$ upscaling factors.
- We propose a perceptual loss function which consists of an adversarial loss and a content loss. The adversarial loss pushes our solution to the natural image manifold using a discriminator network that is trained to differentiate between the super-resolved images and original photo-realistic images

SRGAN

Generator Network



Discriminator Network



SRGAN



Figure 2: From left to right: bicubic interpolation, deep residual network optimized for MSE, deep residual generative adversarial network optimized for a loss more sensitive to human perception, original HR image. Corresponding PSNR and SSIM are shown in brackets. [4× upscaling]

SRGAN

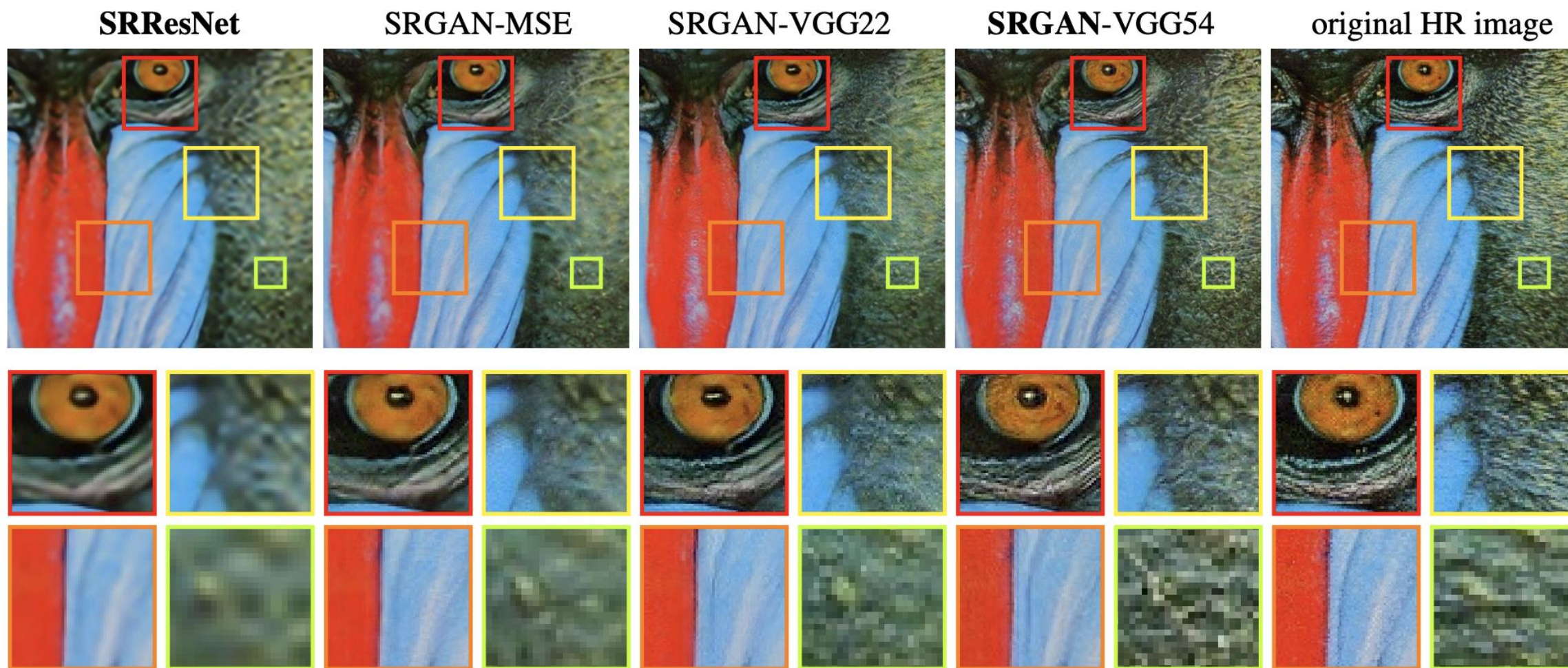


Figure 6: **SRResNet** (left: a,b), **SRGAN-MSE** (middle left: c,d), **SRGAN-VGG2.2** (middle: e,f) and **SRGAN-VGG54** (middle right: g,h) reconstruction results and corresponding reference HR image (right: i,j). [4× upscaling]

Summary

